

合肥工业大学

《计算机应用项目实训》 智能无人飞行器设计报告

基于视觉识别、云端组网与自动投放的
智能无人飞行器系统设计与实现

学生姓名	朱清扬
学号	2023212290
班级	计科23-3班
小组名称	马刺总冠军
个人分工	目标识别、自动投放、云端组网与4G图传
验收地点	东操场
完成日期	2026年6月11日

摘要

智能无人飞行器综合实训涉及固定翼飞行平台、飞控系统、嵌入式计算、计算机视觉、机器人中间件、跨公网通信和机电执行机构等多个技术方向。本项目以塞斯纳固定翼无人机为载体，完成了机体与飞控配置、泰山派 RK3566 机载计算平台部署、坦克目标识别模型训练、ROS 与 MAVROS 任务程序、WireGuard 三端虚拟专网、4G 实时图像回传、目标地理坐标解算及自动投弹控制等功能。系统由 Pixhawk 飞控负责姿态稳定、航线执行和飞行状态输出，泰山派负责摄像头采集、YOLO/RKNN 推理和任务逻辑，阿里云服务器负责跨公网数据中继，Ubuntu 虚拟机负责地面端图像显示与终端监控，投弹舵机负责执行最终释放动作。

目标识别部分采用 YOLOv5s 预训练模型进行单类别坦克检测。实际保存的训练记录中，exp5 使用 640 像素输入尺寸、批大小 16 和 300 轮配置，并因早停机制在第 178 轮附近结束。末轮精确率约为 0.958，召回率约为 0.963， $mAP@0.5$ 约为 0.950， $mAP@0.5:0.95$ 约为 0.411。模型通过 ONNX 与 RKNN 工具链转换后部署到 RK3566，检测结果以 ROS 消息形式提供给目标定位节点。目标定位程序综合图像中心偏移、飞行高度、航向角和无人机 GPS 位置估算目标经纬度，并通过范围约束和多帧聚合降低单帧检测波动。

跨公网通信部分将阿里云服务器、地面虚拟机和泰山派分别配置为 10.0.0.1、10.0.0.2 和 10.0.0.3。服务器开放 UDP 51820 端口、启用 IPv4 转发并配置防火墙转

发规则，客户端通过 PersistentKeepalive 维持运营商 NAT 映射。ROS 节点统一使用 WireGuard 地址注册，使图像话题能够从泰山派经 4G 网络传输到地面虚拟机。自动投弹程序依据飞行高度计算自由落体时间，根据水平速度和目标距离计算释放提前量，并加入通信、程序和舵机动作补偿；执行端通过 20 ms 周期的 PWM 信号控制舵机完成挂钩释放。

外场验收在东操场进行。实飞过程表明固定翼无人机能够正常升空和飞行，地面虚拟机能够接收机载实时图像，语音节点能够识别任务指令并推送航点，目标识别、通信链路和飞行任务能够在同一系统中协同工作。项目验证了计算机视觉、嵌入式系统、计算机网络和飞行控制技术在固定翼无人机任务中的综合应用。

关键词： 固定翼无人机；YOLOv5；RK3566；ROS；MAVROS；WireGuard；4G 图传；自动投弹

目录

课题概述

课题任务

技术方案及关键问题

设计实现及测试

课程设计总结

参考文献

1. 课题概述

1.1 课题背景

无人飞行器是飞行平台、导航控制、通信链路、嵌入式计算、任务载荷和人工智能算法的综合载体[1]。传统航模主要依靠人工遥控完成飞行，任务执行高度依赖操作者经验；普通无人机即使具备飞控，也常停留在定点、定高和航线飞行层面，缺乏面向具体目标的自主感知与决策能力。随着嵌入式处理器、轻量化神经网络、卫星定位、移动通信和机器人中间件的发展，无人机可以在有限载荷和功耗条件下完成“感知、传输、判断、执行”的闭环任务。

本课题来源于《计算机应用项目实训：智能无人飞行器》课程。课程以塞斯纳固定翼模型为对象，把计算机专业中的操作系统、计算机网络、嵌入式系统、人工智能、程序设计和软件工程知识放入同一工程场景。项目并非只验证某一个算法，而是要求多个异构模块协同工作：飞控负责稳定飞行，泰山派负责边缘计算，摄像头提供视觉输入，YOLO 模型检测目标，MAVROS 提供飞行状态，4G 与 VPN 提供跨公网链路，地面虚拟机负责监视和远程调试，舵机完成最终动作。任何一个环节配置错误，都会表现为最终功能不可用。

固定翼平台相较于多旋翼具有航程长、巡航效率高和续航时间长等特点，适合执行大范围巡查、测绘和搜索任务，但不能像多旋翼那样原地悬停，起降和航迹控制要求更高。自动投弹任务进一步放大了这一特点：程序不能等飞机到达目标正上方才释放，

而要根据飞行速度、高度和系统延迟计算提前量。因此，本项目既包含目标检测问题，也包含实时通信、坐标变换、任务规划和机电执行问题。

1.2 课题目的

本课题的直接目的，是完成一套可运行的智能固定翼无人飞行器原理样机，并用可复现的实验过程证明各模块之间能够协同。具体而言，需要理解固定翼升力、阻力和舵面控制原理；掌握飞控固件、传感器校准、接口映射和自主航线配置方法；掌握 RK3566 系统镜像、ROS 工作空间和摄像头节点的部署方法；掌握 YOLO 数据集制作、训练、评估、转换和边缘端推理流程；掌握 WireGuard 在 NAT 和移动网络环境中的组网、路由与保活机制；掌握 MAVROS 话题订阅、目标坐标计算和 PWM 舵机控制方法。

从计算机专业能力的培养角度看，本课题的价值在于建立系统化排障意识。单机程序常可通过调试器定位问题，而无人机系统的问题可能同时来自电源、接线、驱动、网络路由、环境变量、ROS 节点、消息类型、模型格式和物理机构。只有把系统分成物理层、网络层、中间件层、算法层和执行层，并为每一层设置可观察的验证点，才能提高联调效率。

1.3 课题意义

本项目将人工智能算法从训练环境迁移到真实嵌入式平台，能够直观展示模型精度、推理速度、网络带宽和飞行安全之间的约束关系。训练精度高并不等于系统一定

可用；如果模型过大导致帧率过低，目标检测结果就会滞后；如果传输原始图像导致 4G 上行拥塞，即使本地识别正常，地面端仍无法实时观察；如果坐标变换忽略航向角，检测框再准确也会得到错误的地理位置；如果舵机供电不足或 PWM 参数不合理，程序日志显示触发也不能完成机械释放。

项目还具有明显的网络工程意义。泰山派通过 4G 接入时通常位于运营商 NAT 后，虚拟机也处于校园网或局域网 NAT 后，两端不能简单依靠私有地址直接通信。使用具有固定公网地址的云服务器作为 WireGuard 中枢，可把三台设备映射到同一虚拟网段，并以加密隧道承载 ROS 控制面和数据面流量。这一方案把复杂公网环境隔离在隧道层，上层 ROS 节点仍可使用稳定的 10.0.0.0/24 地址通信。

2. 课题任务

2.1 总体任务

课题总体任务是设计并实现一套具备自主飞行、目标识别、跨公网图传、语音任务触发和自动投弹能力的固定翼无人机系统。系统应形成从任务输入到物理动作的闭环：地面端或语音节点给出任务指令，航线推送程序将航点写入飞控，飞控控制飞机沿预定航迹飞行，机载摄像头采集地面图像，泰山派运行 YOLO 模型输出目标检测结果，目标定位程序融合像素位置和飞行状态估算目标经纬度，投弹程序计算释放时机并控制舵机，地面虚拟机通过 4G VPN 实时接收图像和运行状态。

2.2 功能任务

完成固定翼机体、动力系统、舵机、飞控、GPS、摄像头、泰山派、4G 模块和导弹机构的装配与整机连接。

完成飞控固件安装、传感器校准、遥控器校准、飞行模式配置、舵面方向检查和自主航线上传。

完成 RK3566 系统烧录、存储扩展、网络接入、ROS 环境和机载工作空间部署。

构建坦克目标单类别数据集，完成 YOLOv5s 模型训练、指标评估、权重导出及 RKNN 嵌入式部署。

配置阿里云服务器、虚拟机和泰山派三端 WireGuard 网络，实现跨公网两两互通和稳定保活。

配置 ROS_MASTER_URI 与 ROS_IP，实现跨设备 ROS 节点发现和图像话题实际传输。

订阅 GPS、相对高度、航向角、速度和目标像素坐标等话题，完成目标地理坐标估算。

根据高度、水平速度、目标距离和执行延迟计算投放提前量，通过 PWM 控制舵机完成释放。

通过地面静态测试、链路测试、语音测试、图传测试和外场实飞对系统进行分层验证。

2.3 技术指标与验收依据

表 2.1 系统功能与验收依据

模块	目标指标	本项目验证依据
飞行平台	可完成固定翼起飞、空中飞行与安全落地	外场实飞连续截图和视频
目标识别	可识别坦克目标并输出检测框和置信度	训练结果文件、best.pt、模型曲线
网络互通	服务器、虚拟机、泰山派两两可达	WireGuard 状态与完整链路截图
实时图传	虚拟机可显示泰山派摄像头 ROS 图像	rqt_image_view 实时画面截图
语音任务	可识别任务口令并触发航点推送	语音识别及航点推送终端截图
自动投弹	程序可计算时机并驱动投弹舵机	代码逻辑、地面释放测试与整机照片
安全性	异常数据不触发投弹，实飞前完成分级验证	阈值判断、范围约束和分层测试记录

本项目以课程综合设计的功能闭环和工程可实现性为主要验收目标。模型训练采用实际保存的训练记录评价，网络与图传通过三端连通、图像显示和任务终端状态进行验证，飞行平台通过外场起飞、空中飞行和地面端实时回传进行综合验收。

3. 技术方案及关键问题

3.1 总体架构

系统采用“飞控实时控制 + 泰山派边缘计算 + 云服务器网络中继 + 虚拟机地面监控”的分层架构。飞控与泰山派通过串口和 MAVLink 协议交换数据，MAVROS 将 MAVLink 消息转换为 ROS 话题与服务；摄像头通过 USB 接入泰山派，YOLO 节点在 RK3566/NPU 环境中进行推理；泰山派、云服务器和虚拟机通过 WireGuard 形成虚拟局域网；图像、检测结果和任务状态通过 ROS 节点传递；投弹程序通过 Linux PWM 接口控制舵机。

无人机任务系统分层架构与数据流程图

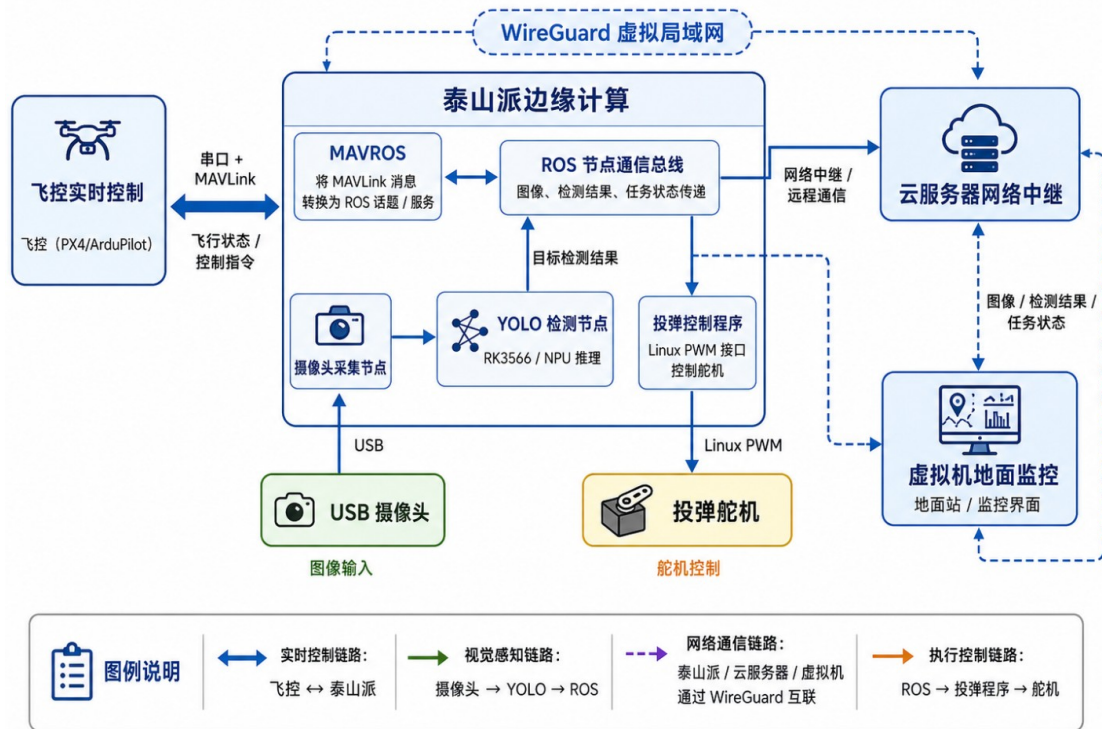


图 3.1 系统总体架构

3.2 飞行平台方案

平台选用塞斯纳布局固定翼模型。机翼产生主要升力，副翼控制滚转，升降舵控制俯仰，方向舵控制偏航，电机和螺旋桨提供前向推力。固定翼在巡航状态下能量效率高，但飞行速度不能降得过低，否则升力不足并可能失速。飞控通过惯性测量单元、气压计、GPS 和磁罗盘估计姿态与位置，再根据遥控输入或自主航点指令输出各通道控制量。

自动任务中采用飞控承担快速闭环、泰山派承担慢速任务决策的分工。姿态稳定要求毫秒级响应，不适合由普通 Linux 用户态程序直接控制；目标识别和任务规划计

算量大、更新频率相对较低，适合放在泰山派。二者通过 MAVROS 解耦，使算法程序只需订阅标准 ROS 消息，而不必直接解析全部 MAVLink 帧。

3.3 目标识别方案对比

目标检测可采用传统特征方法、两阶段深度学习或单阶段深度学习。传统方法依赖人工设计特征，对尺度、光照和背景变化敏感；Faster R-CNN 等两阶段方法精度较高，但模型复杂、推理开销大；YOLO 系列将检测转化为端到端回归，速度和精度平衡较好，便于嵌入式部署[3]。本项目训练包和 RKNN 部署链均以 YOLOv5 为基础，因此选用 YOLOv5s。

YOLOv5s 参数量和模型体积较小，工作区中的 best.pt 约为 14 MB，适合从 PC 训练环境转换后部署到 RK3566。检测评价采用精确率、召回率和平均精度。精确率表示预测为目标的样本中有多少是真目标，召回率表示真实目标中有多少被检出：

$$P = \frac{TP}{TP + FP} \quad (3.1)$$

$$R = \frac{TP}{TP + FN} \quad (3.2)$$

当交并比阈值设为 0.5 时，可计算 mAP@0.5；对 0.5 至 0.95 的多个阈值取平均均可得到更严格的 mAP@0.5:0.95。对投弹任务而言，漏检会导致任务无法触发，误检会产生安全风险，因此不能只看单一指标，还需要设置地理范围、连续帧数量和状态有效性等二次约束。

3.4 跨公网通信方案对比

直接使用 4G 模块获得的地址通常不能被公网主动访问，因为运营商会采用大规模 NAT；校园网和宿舍网络也常存在多层 NAT。端口映射依赖网络管理权限，不适用于移动网络；传统反向代理适合特定 TCP 服务，但对 ROS 多节点动态端口支持不方便；WireGuard 能在内核态建立三层虚拟网络，配置项少、加密开销低，并可通过云服务器中继解决两端都位于 NAT 后的问题[5]。

本项目将云服务器设为 10.0.0.1/24，虚拟机设为 10.0.0.2/24，泰山派设为 10.0.0.3/24。客户端主动连接服务器的固定公网 IP 和 UDP 51820 端口。服务器开启 IPv4 转发并允许 wg0 到 wg0 的转发后，可以在两个客户端之间中继数据。PersistentKeepalive = 25 用于维持 NAT 映射，减少空闲后链路失效。

3.5 ROS 跨主机方案

ROS Master 只负责节点注册与连接发现，不负责转发图像数据[2]。虚拟机能够执行 rostopic list 只能证明控制面可达；订阅者获得发布者地址后仍需与发布者直接建立 TCPROS 或其他传输连接。如果泰山派向 Master 注册的是 192.168.x.x 等物理网卡地址，虚拟机即使能访问 Master，也不能访问该私网地址，最终表现为“能看到话题但没有图像”。

因此泰山派配置：

```
export ROS_MASTER_URI=http://10.0.0.3:11311
```

```
export ROS_IP=10.0.0.3
```

虚拟机配置：

```
export ROS_MASTER_URI=http://10.0.0.3:11311
```

```
export ROS_IP=10.0.0.2
```

两端将配置写入 `~/bashrc` 后执行 `source ~/.bashrc`。图像节点使用 `/rknn_image/theora` 等压缩传输，避免原始图像占满 4G 上行带宽。

3.6 目标定位与投弹方案

目标定位以图像中心为原点计算目标像素偏移，再根据相机视场、图像尺寸和无人机相对高度估算地面平面位移。平面位移需结合无人机航向角旋转到北东坐标系，最后根据当前位置纬度换算经纬度增量。设像素坐标为 (u, v) ，图像中心为 (c_x, c_y) ，则归一化偏移可写为：

$$x_c = \frac{u - c_x}{f_x} h, y_c = \frac{c_y - v}{f_y} h \quad (3.3)$$

其中 h 为相对高度， f_x 和 f_y 为等效焦距参数。若航向角为 ψ ，则相机平面位移旋转到地理坐标系：

$$\begin{bmatrix} x_e \\ y_n \end{bmatrix} = \begin{bmatrix} \cos \psi & -\sin \psi \\ \sin \psi & \cos \psi \end{bmatrix} \begin{bmatrix} x_c \\ y_c \end{bmatrix} \quad (3.4)$$

在小范围近似下，纬度方向每度距离近似稳定，经度方向每度距离与纬度余弦有

关。程序还需对目标经纬度设置允许范围，并对三秒内超过阈值数量的检测结果求平均，降低单帧误差。

投弹模型采用水平匀速和竖直自由落体近似。若飞行高度为 h ，重力加速度为 g ，则理论下落时间为：

$$t_f = \sqrt{\frac{2h}{g}} \quad (3.5)$$

若飞机水平速度为 v ，理论提前距离为：

$$d_f = v t_f \quad (3.6)$$

程序也可根据当前位置到目标的水平距离 d 计算等待时间：

$$t_d = \frac{d}{v} - t_f - t_c \quad (3.7)$$

其中 t_c 为通信、程序和舵机动作的综合补偿时间。本人调试记录中采用约 120 ms 的经验补偿，并在速度低于 0.2 m/s 时跳过除法和投弹判断，防止出现无穷大或非数。

3.7 开发平台与工具链

开发平台由 Windows 训练主机、Ubuntu 虚拟机、阿里云 Linux 服务器、泰山派 RK3566 和 Pixhawk 飞控构成。Windows 主机用于数据标注、YOLOv5 训练和曲线分析；Ubuntu 虚拟机作为 ROS 地面端和远程调试终端；阿里云服务器承担 WireGuard 中继；泰山派运行摄像头、RKNN 推理、MAVROS 任务节点和投弹控制程序；Mission Planner 用于飞控校准、参数检查和航点规划。该工具链把模型训练、

嵌入式部署、网络通信和飞行控制分开，便于逐层验证。

表 3.3 开发平台与主要用途

平台或工具	主要用途	选择依据
YOLOv5 7.0	坦克目标训练与评估	训练资料完整，便于导出 ONNX
RKNN Toolkit	模型转换与 NPU 部署	与 RK3566 芯片匹配
ROS 与 MAVROS	节点通信和飞控数据访问	统一消息、话题和服务接口
WireGuard	三端虚拟专网	配置简洁，适应 NAT 与移动网络
Mission Planner	飞控配置和航线任务	支持 ArduPlane 参数及任务管理
VMware Ubuntu	地面监控和远程调试	与 ROS 工具链兼容

3.8 关键问题与总体解决思路

网络层问题采用从端口、握手、路由、转发到保活的顺序排查，避免在 ROS 层反复重启。

ROS 问题分别检查 Master 可达性、节点注册地址、话题类型和实际频率，区分控制面与数据面。

模型问题分别检查数据集路径、类别数、训练曲线、权重格式和 RKNN 芯片类型，避免把转换失败误判为摄像头故障。

投弹问题将算法计算、PWM 输出、舵机供电和机械释放分开测试，并设置速度、范围和数据有效性门限。

实飞风险通过数据回放、地面静态、低速手持和外场实飞四级验证控制，不直接用未经验证的代码上机。

4. 设计实现及测试

4.1 系统总体设计与实现路线

本项目的实现遵循“先建立稳定飞行平台，再完成机载计算与网络基础，随后部署视觉和任务算法，最后进行整机联调与外场验证”的顺序。采用这一顺序的原因是各模块存在明确依赖关系：目标识别依赖摄像头和嵌入式运行环境，目标坐标计算依赖模型输出和飞控状态，4G 图传依赖 VPN 与 ROS 地址配置，自动投弹依赖目标位置、飞行状态和舵机驱动同时有效。若在底层条件未确认前直接进行整机测试，任一故障都会表现为最终任务失败，难以判断问题所在。

系统总体结构如图 4.1 所示。飞控承担固定翼姿态与航迹的快速闭环控制，泰山派承担图像处理和任务决策。摄像头图像进入 YOLO/RKNN 节点后得到目标类别、置信度与检测框；MAVROS 同时提供 GPS、相对高度、航向角和水平速度；目标定位节点将视觉结果与飞行状态融合，得到目标地理坐标；投弹节点根据目标距离、飞行速度、下落时间和执行补偿判断释放时机；WireGuard 网络把机载端、云服务器和地面虚拟机连接在同一虚拟网段，地面端能够查看图像和任务状态。

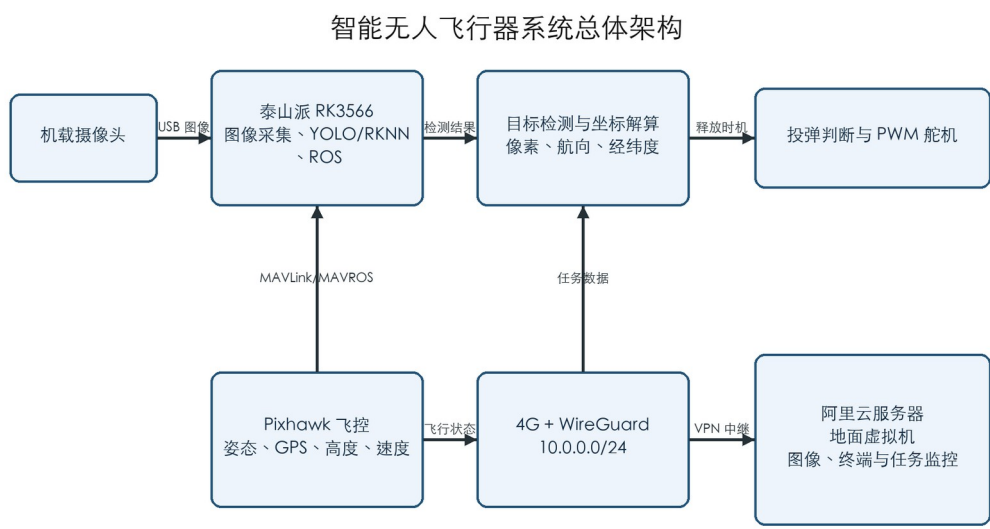


图 4.1 智能无人飞行器系统总体架构

项目实施过程划分为八个阶段，如图 4.2 所示。每个阶段均设置独立的验收点：

机体阶段检查舵面和重心，飞控阶段检查传感器与 MAVROS，泰山派阶段检查设备和系统资源，模型阶段检查训练曲线和推理输出，网络阶段检查握手与路由，ROS 阶段检查节点和话题，投弹阶段检查数值与 PWM，最终通过地面测试和外场实飞验证整体链路。

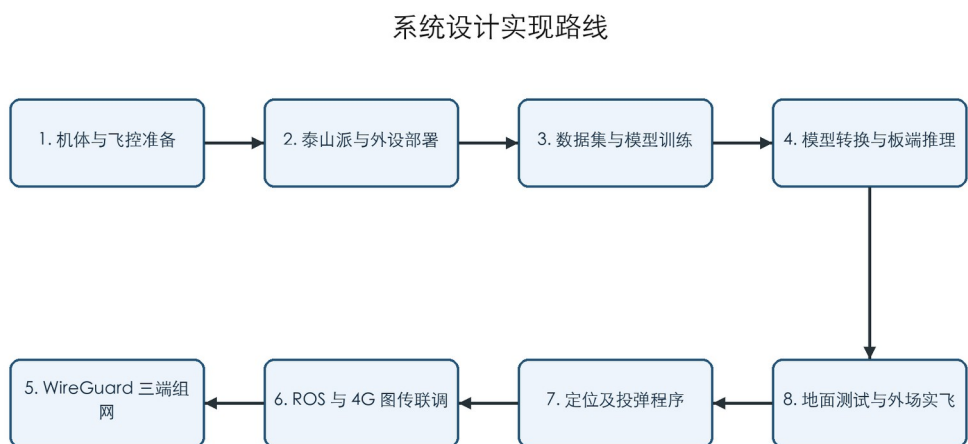


图 4.2 系统设计实现路线

表 4.1 系统实施阶段及主要验收点

实施阶段	主要工作	验收点
飞行平台	机体、舵面、动力、重心与飞控安装	舵面方向正确，飞控状态稳定
机载平台	泰山派、摄像头、4G 模块和舵机连接	设备节点存在，供电和散热正常
模型训练	数据标注、训练、评估和权重选择	曲线收敛，验证集检测结果正确
模型部署	ONNX、RKNN 转换及板端推理	板端能够输出目标检测结果
网络组网	云服务器与两客户端 WireGuard 配置	三端地址唯一且两两可达
ROS 图传	Master、节点地址和压缩图像配置	地面端持续显示机载画面
任务程序	坐标解算、投弹判断和 PWM 控制	数值有效，舵机独立动作正常
综合测试	地面分级验证与外场实飞	起飞、图传、任务终端同时工作

4.2 实验平台与开发环境

实验平台由固定翼飞行平台、Pixhawk 飞控、泰山派 RK3566、USB 摄像头、4G 通信模块、投弹舵机、阿里云服务器和 Ubuntu 虚拟机组成。Pixhawk 运行 ArduPlane 固件，负责姿态估计、舵面输出、自主航线和飞行状态采集。泰山派运行 Linux 与 ROS 环境，通过串口与飞控通信，通过 USB 获取图像和 4G 网络，通过 PWM 引脚控制投弹舵机。地面虚拟机承担 ROS 图形工具、远程终端和任务状态观察。训练主机运行 YOLOv5 7.0、PyTorch 和 CUDA 环境，保存数据集、训练日志、权重及评估曲线。

表 4.2 实验软硬件平台

类别	平台或软件	在系统中的作用
飞行平台	塞斯纳固定翼模型	承载动力、飞控、任务计算机和载荷
飞控	Pixhawk / ArduPlane	姿态控制、GPS 定位、航线和状态输出
机载计算机	泰山派 RK3566	ROS、图像采集、RKNN 推理和投弹程序
视觉传感器	USB 摄像头与云台	采集地面目标图像
通信	4G 模块、阿里云、WireGuard	构建跨公网虚拟局域网
地面端	VMware Ubuntu 虚拟机	远程调试、图像显示和任务监控
模型工具	YOLOv5 7.0、PyTorch、RKNN	训练、导出、转换与板端部署
机器人中间件	ROS、MAVROS	统一飞控、视觉和任务节点接口

机载软件的数据流如图 4.3 所示。摄像头节点和 YOLO 节点构成视觉前端，MAVROS 构成飞行状态前端，目标定位和投弹控制构成任务决策层，WireGuard 与地面虚拟机构成监控链路。各层通过标准 ROS 话题和函数接口连接，便于单独启动和验证。

机载软件与 ROS 数据流

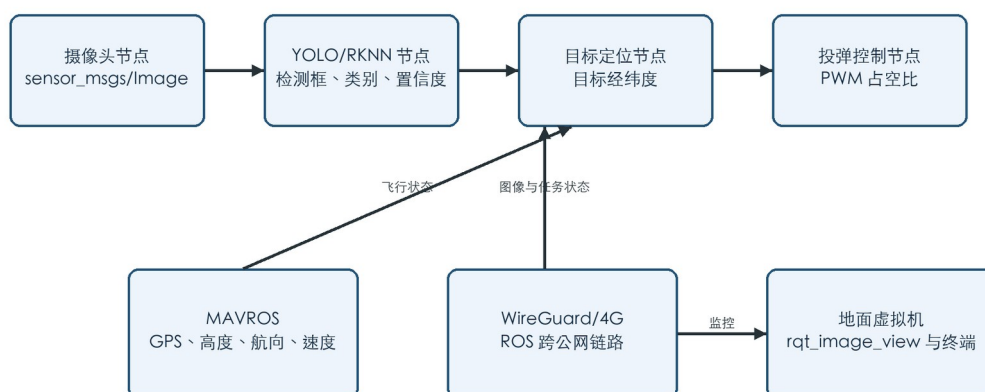


图 4.3 机载软件与 ROS 数据流

4.3 固定翼机体、飞控及任务载荷

固定翼飞机依靠机翼上下表面的压力差产生升力。设空气密度为 ρ ，飞行速度为 v ，机翼面积为 S ，升力系数为 C_L ，则升力可表示为：

$$L = \frac{1}{2} \rho V^2 S C_L \quad (4.1)$$

固定翼不能像多旋翼一样原地悬停，速度过低会使升力下降并增加失速风险，因此机体装配必须保证左右机翼对称、舵面活动顺畅、连接杆间隙合理和重心位置正确。副翼用于滚转控制，升降舵用于俯仰控制，方向舵用于偏航控制。安装完成后先拆除螺旋桨检查电机转向和各舵面响应，再在低风险条件下进行滑跑和起飞准备。

飞控安装在接近机体重心的位置，箭头方向与机头一致，并通过减振材料降低电机振动对惯性测量单元的影响。GPS 和磁罗盘远离电调、电机和动力线。通过 Mission Planner 完成加速度计、罗盘、遥控器和飞行模式校准，并核对 MAIN OUT 各通道与副翼、升降舵、油门和方向舵的映射。TELEM 串口用于飞控与泰山派之间的 MAVLink 通信，飞控状态由 MAVROS 转换为 ROS 话题。

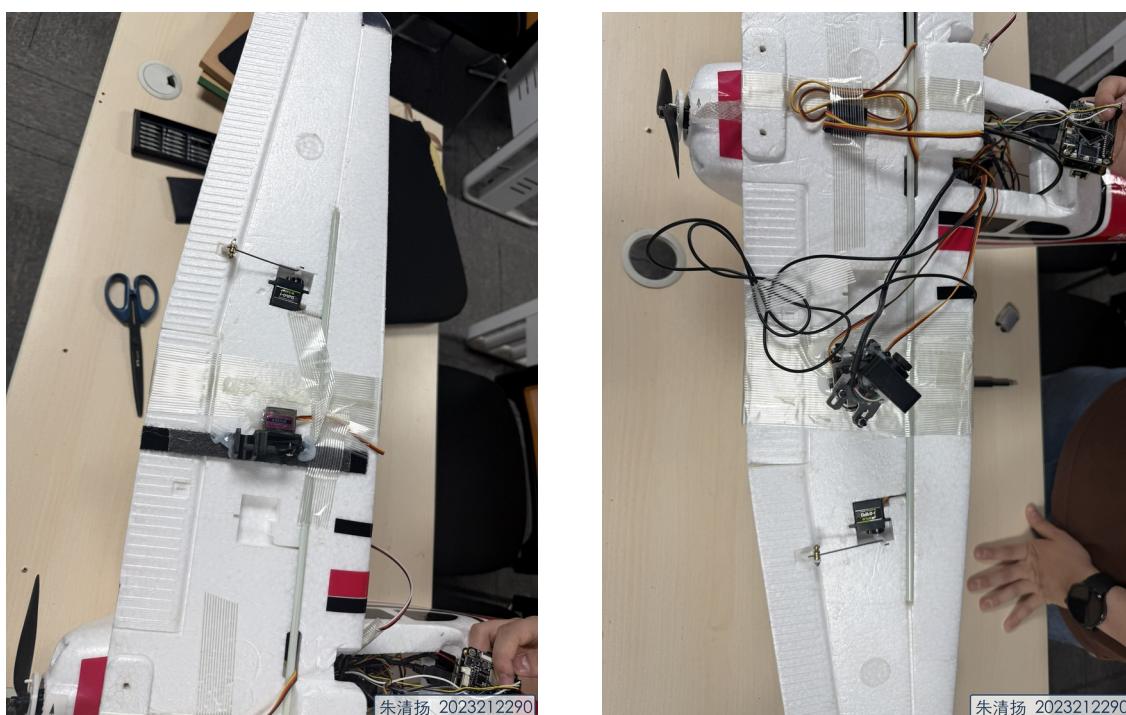


图 4.4 固定翼平台两侧任务载荷的实际安装

图 4.4 中，机翼一侧安装云台摄像头，另一侧安装投弹舵机和释放机构，使两侧附加载荷尽量保持平衡。机舱内部布置飞控、泰山派、供电和通信线束。安装时不仅需要保证器件固定，还需要避免线束干涉舵机拉杆，并为 USB、串口和电源接口预留应力缓冲。



图 4.5 Pixhawk 飞控及机舱线束实际状态

4.4 机载硬件连接与调试

泰山派是机载任务计算核心，USB 摄像头提供图像，Pixhawk 通过 TELEM 串口提供飞行状态，4G 模块提供公网接入，投弹舵机由 PWM8_M0 控制。硬件连接关系如图 4.6 所示。串口连接需要交叉连接 TX 与 RX 并共地；摄像头和 4G 模块占用 USB 或对应通信接口；PWM 线、5 V 电源和 GND 分别连接舵机信号、电源和地。

舵机负载电流较大时，使用独立 BEC 供电并保持控制地与泰山派共地，可避免舵机启动造成板端电压下降。

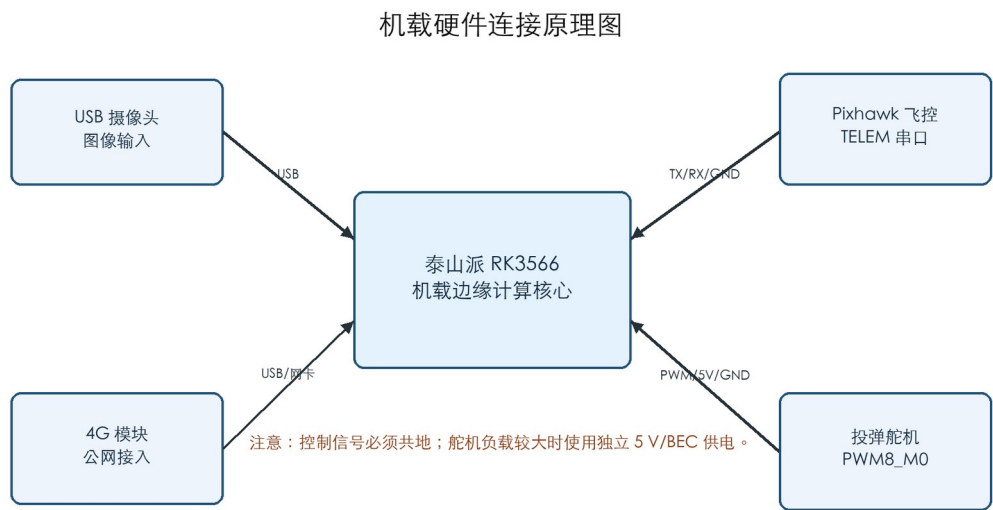


图 4.6 机载硬件连接原理图

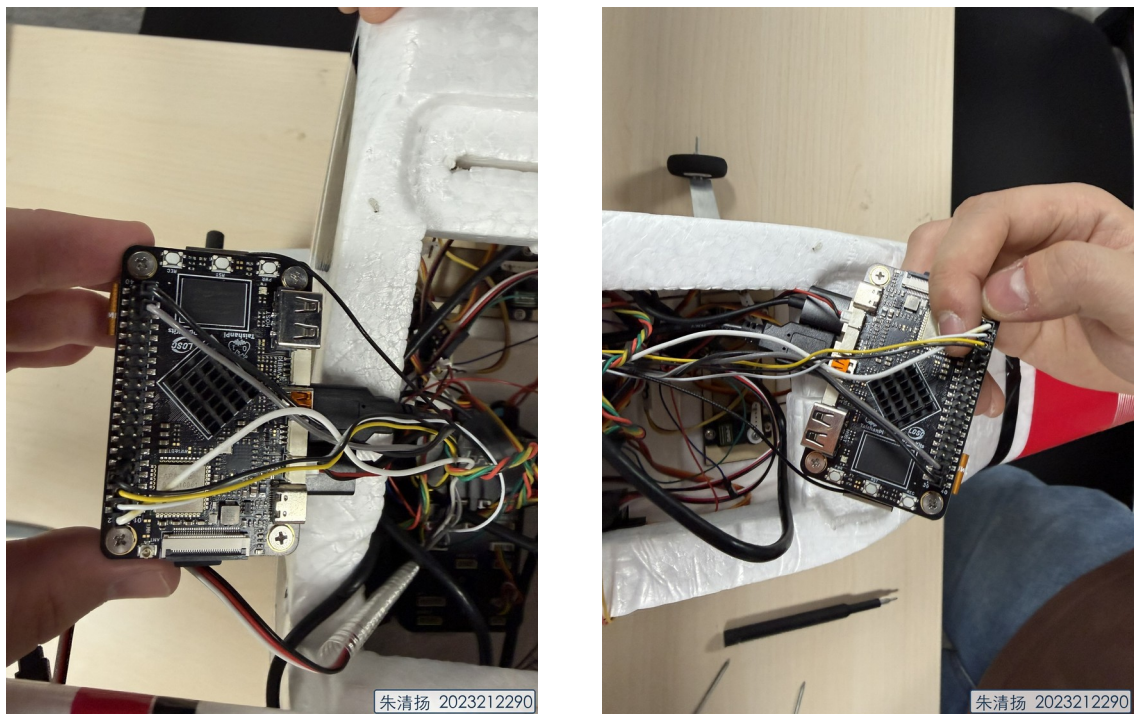


图 4.7 泰山派与机舱线束实际安装状态

硬件调试采用由静态到动态的顺序。首先使用 `lsusb`、`ls /dev/video*`、`dmesg` 和串口输出确认设备被系统识别；其次检查文件系统剩余空间、CPU 温度和电源稳定性；随后分别启动摄像头、MAVROS 和 PWM 测试程序；最后再启动 YOLO、网络图传和任务程序。这样能够把硬件识别、驱动、系统资源和应用程序问题分开。

机载平台运行问题及处理

系统磁盘空间不足

在该环节的首次联调中，出现的主要现象如下。在泰山派上编译 ROS 工作空间，或者下载语音识别的大模型文件时，终端突然持续报错说 `No space left on device`（磁盘空间不足），然后系统就开始变得极度卡顿，连 SSH 都快连不上了。

结合命令输出、节点状态和接口工作过程进行检查后，原因可归纳如下。泰山派开发板自带的 `emmc` 存储空间非常小，刷完系统镜像后，留给用户的可用空间所剩无几。而本组实训需要编译大量的 C++ ROS 包，还要存放 YOLOv5 模型和语音识别模型，这些文件动辄几百兆。如果本组把这些大文件和编译空间都直接放在默认的家目录下，系统盘随后就会被塞满。

根据上述原因，对系统进行了以下调整与验证。

1. 本人插上了一张 32G 的 TF 卡（SD 卡），并将其格式化为 Linux 的 `ext4` 分区格式。
2. 用 `df -h` 查看挂载情况，把 TF 卡挂载到 `/mnt/sdcard` 或者家目录下的一个专用文件夹（比如 `~/workspace`）里。
3. 本人把所有的 ROS 源码工作空间、下载模型文件、大体积的数据集统统移动到 TF 卡对应的目录下，并在 `~/.bashrc` 里把工作空间路径配置好。
4. 定期运行 `sudo`

apt-get clean 和清理 `~/ros/log/` 下的 ROS 运行日志。这样系统盘的占用就一直维持在安全线以下，再也没有报过空间不足。

高温导致开发板降频或节点卡死

在该环节的首次联调中，出现的主要现象如下。本组把无人机拿到操场上联调时，刚启动的时候一切正常。但是当飞机在太阳底下晒了一会儿，且本组同时运行着摄像头、YOLO 识别和 4G 传输后，泰山派的系统反应就变得越来越慢，敲个命令要卡好几秒，甚至 YOLO 推理的帧率直接跌到了 1、2 帧，最后程序直接无响应卡死。

结合命令输出、节点状态和接口工作过程进行检查后，原因可归纳如下。泰山派板载的 RK3566 芯片在满载运行深度学习模型（NPU）和图像编解码时，发热量非常大。如果在室外高温下，且没有任何散热措施，芯片温度很容易飙升到 80°C 以上。为了防止芯片烧毁，Linux 内核的温度保护机制会自动限制 CPU 和 NPU 的最高运行频率（也就是主动降频）。算力一掉，需要高实时性的 ROS 节点就会因为处理不过来数据而排队堵塞，最终导致系统卡死。

根据上述原因，对系统进行了以下调整与验证。1. 本人给泰山派的 CPU/NPU 芯片贴上了导热硅胶垫，并加装了一块铝制被动散热片。2. 本组还在无人机机身侧面用 3D 打印设计了一个小支架，装上了一个 5V 的超薄散热小风扇，直接从动力电池的 BEC 5V 引脚供电，对准板子进行强风降温。3. 在调试和验收前，本人关掉了所有不需要的系统后台服务（比如图形化桌面服务，只用 SSH 命令行模式跑），尽量

减轻系统的运算负担。经过这些改造，即使在室外大太阳下联调，芯片温度也一直稳定在 55°C 左右，运行保持稳定运行。

镜像烧录过程中断

在该环节的首次联调中，出现的主要现象如下。使用 RKDevTool 烧录工具往泰山派烧录实验镜像时，进度条刚走到百分之六七十，电脑就突然弹窗报错提示写入失败。重新给板子上电后，板子上的指示灯不正常，串口也没有任何输出，设备彻底“砖”了，电脑的烧录工具也再也识别不到 Loader 设备。

结合命令输出、节点状态和接口工作过程进行检查后，原因可归纳如下。烧录中断通常是由于物理连接不稳定或者供电不足引起的。有些同学是用笔记本电脑的 USB 口直接给开发板供电加烧录，由于板子在烧录写入时电流会激增，如果 USB 口输出功率不够，就会导致芯片短暂掉电复位，从而打断烧录。另外，如果烧录过程中电脑因为长时间没有操作进入了休眠状态，USB 接口被系统自动关闭，也会导致烧录中断，损坏底层的引导数据（Bootloader）。

根据上述原因，对系统进行了以下调整与验证。1. 在重新烧录前，本人先把电脑的系统休眠和屏幕关闭时间设置为“从不”，防止烧录时系统休眠。2. 拔掉板子上的其他高耗电设备（如 4G 模块、摄像头），使用短且粗的优质数据线连接电脑的 USB 3.0 端口（蓝色接口），确保供电充足。3. 因为底层的引导文件已经损坏，常规的按键进入 Loader 模式已经失效。本人必须使用“Maskrom 模式”来强行恢复。本人在断电状态下，用镊子短接板子上的 Maskrom 两个触点（对照官方引脚图），同时插上

USB 线供电。 4. 看到烧录工具界面显示“发现一个 Maskrom 设备”后，本人松开镊子。先烧录一个官方的修复用 MiniLoader 支持文件，把底层引导修复好，然后重新导入本组的实验镜像进行完整烧录，直到显示“烧录成功”并自动重启进入系统。

使用不合适的数据线导致设备识别异常

在该环节的首次联调中，出现的主要现象如下。 本人按照课件上的要求，按住泰山派的 REC 键不松，然后去按 RST 重启键，想要让板子进入 Loader 模式烧录。但是不管本人怎么重复这个按键动作，电脑上的烧录工具一直显示“没有发现设备”，毫无反应。

结合命令输出、节点状态和接口工作过程进行检查后，原因可归纳如下。 本人一开始以为是开发板坏了，后来才发现是本人用的那根 Type-C 数据线有问题。现在市面上很多手机充电线为了省成本，内部只接了电源线，根本没有接数据线（Data lines），这种线只能用来充电，没法传输数据。另外，有些线虽然是数据线，但是因为线径太细、内阻太大，导致信号衰减严重，无法在高速烧录模式下被电脑识别。

根据上述原因，对系统进行了以下调整与验证。 1. 本人找队友借了一条原装的、确认可以进行手机数据传输的优质 Type-C 数据线。 2. 重新连接后，再次进行按键操作：先按住 REC 键不要松开，接着按 RST 键，等待大约 2 秒钟，然后松开 REC 键。 3. 这时，电脑的 RKDevTool 烧录工具下方立刻跳出了“发现一个 LOADER 设备”的提示，设备成功被识别。这提醒本组，硬件调试时一定要准备几根靠谱的数据线，避免在低级问题上浪费时间。

开发板无法进入 Loader 模式

在该环节的首次联调中，出现的主要现象如下。数据线没问题，按键操作也是对的，但是板子无论如何都进不去 Loader 模式。每次一上电，它就直接去运行之前写坏了的系统，然后卡死在某个启动画面，烧录工具始终识别不到设备。

结合命令输出、节点状态和接口工作过程进行检查后，原因可归纳如下。这种属于比较严重的底层数据损坏。由于板子内部 Flash 里的引导程序（U-Boot）或者系统内核已经被本组之前写坏的代码污染了，它在上电引导时会卡在半路，导致芯片无法响应常规的 REC 键中断请求，从而无法通过按键进入 Loader 烧录状态。

根据上述原因，对系统进行了以下调整与验证。针对这种“死锁”状态，有效处理方式就是跳过 Flash 的引导，直接让 CPU 进入芯片出厂自带的底层 ROM 恢复模式（即 Maskrom 模式）。1. 本人先让泰山派完全断电（拔掉所有 USB 线和电源）。2. 对照瑞芯微板子的官方引脚原理图，在板子上找到了标有 CLK 和 GND（或者专门的 MASKROM）的两个小金属触点。3. 本人用一个小金属片（如曲别针或镊子）将这两个触点牢牢短接在一起，在保持短接的状态下，插上 USB 线连接电脑。4. 听到电脑发出设备接入的提示音，且烧录工具上显示“发现一个 MASKROM 设备”后，本人移开镊子。5. 此时芯片处于最原始的接收状态，本人依次导入 Loader 文件和系统镜像，点击执行烧录，成功把系统重新刷了进去。

4.5 ROS 与 MAVROS 通信基础

MAVLink 是飞控与外部计算机之间的轻量通信协议，MAVROS 将 MAVLink 消

息转换为 ROS 话题、服务和参数接口。飞控实时发布位置、速度、姿态、相对高度和系统状态，任务程序无需直接解析串口字节流，只需订阅相应话题。常用输入包括 /mavros/global_position/global、/mavros/global_position/rel_alt、/mavros/global_position/compass_hdg、/mavros/local_position/velocity_local 和 /mavros/state。

启动 MAVROS 前先确认飞控串口设备和波特率。启动后依次检查连接状态、消息频率和数值范围：

```
roslaunch mavros apm.launch fcu_url:=/dev/ttyUSB0:57600
```

```
rostopic echo -n 1 /mavros/state
```

```
rostopic hz /mavros/global_position/global
```

```
rostopic echo -n 1 /mavros/global_position/rel_alt
```

```
rostopic echo -n 1 /mavros/global_position/compass_hdg
```

/mavros/state 中 connected 为真说明 MAVLink 链路建立；GPS 和高度话题持续更新说明导航数据可供任务程序使用。若仅能看到节点而状态不更新，应检查串口占用、设备权限、波特率和飞控 TELEM 参数，而不是直接修改上层算法。

4.6 数据集制作与 YOLO 格式检查

目标数据集采用单类别 YOLO 格式，类别名称为 tank。每张图片对应一个同名文本文件，每一行包含类别编号、目标中心横坐标、目标中心纵坐标、目标宽度和目标高度。设图像宽高分别为 W 和 H ，标注框左上角为 (x_1, y_1) 、右下角为 (x_2, y_2) ，则归一化标签为：

$$x_c = \frac{x_1 + x_2}{2W}, y_c = \frac{y_1 + y_2}{2H} \quad (4.2)$$

$$w = \frac{x_2 - x_1}{W}, h = \frac{y_2 - y_1}{H} \quad (4.3)$$

所有坐标应位于 0 至 1 之间。数据集制作流程包括图像筛选、目标标注、标签检查、训练集与验证集划分、配置文件编写和训练前扫描，如图 4.8 所示。

YOLO 数据集构建与模型训练流程



图 4.8 YOLO 数据集构建与模型训练流程

数据集配置文件 `ccsszz.yaml` 指定数据根目录、训练集、验证集、类别数和类别名称：

```

path: datasets

train: images/train

val: images/val

nc: 1

names: [tank]
  
```

训练前使用脚本检查图片与标签是否一一对应，读取每个标签的列数和坐标范围，

并抽样显示标注框。数据分布图显示数据集中以单目标样本为主，目标尺寸和位置存在一定变化。验证集预测图能够直观检查模型是否把坦克目标框在合理位置。

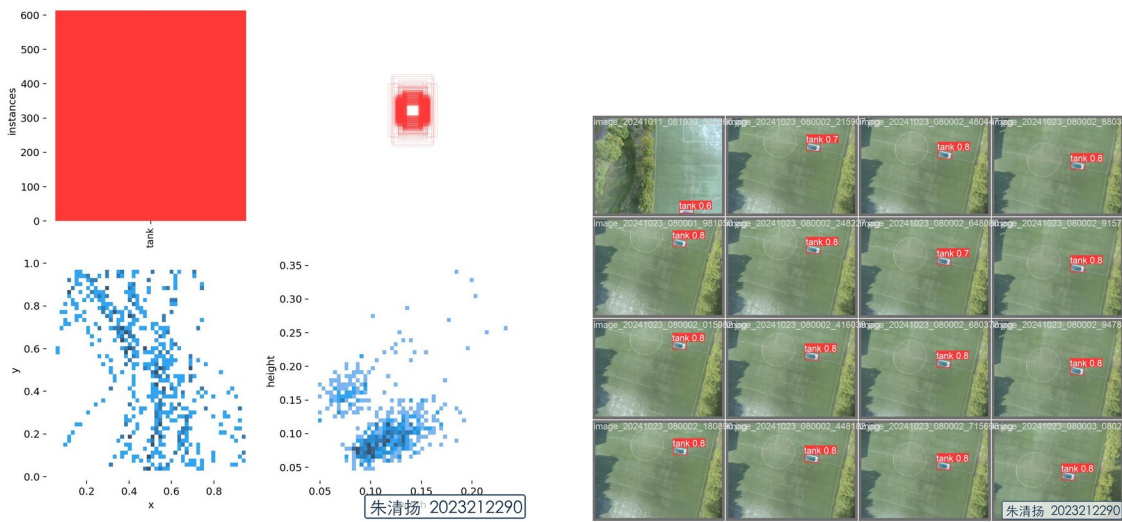


图 4.9 数据集分布与验证集实际预测结果

数据集检查过程中发现的问题及处理

数据集重复、模糊，导致模型泛化能力较差

在该环节的首次联调中，出现的主要现象如下。本人在电脑上训练 YOLOv5，看训练日志里 Precision 和 Recall 两个指标都接近 95% 以上，表现非常完美。但是，当本人把模型拿到实际的摄像头前去测，或者换个房间、换个光照环境时，它对坦克的识别就变得极其迟钝，经常漏检，或者把垃圾桶、椅子脚也误认成坦克。

结合命令输出、节点状态和接口工作过程进行检查后，原因可归纳如下。本人回头去翻了翻本组采集的数据集，发现了问题所在。本组之前的很多照片都是在同一个背景、同一个角度连续拍的（比如按着相机快门连拍）。这导致图片中背景占比大，且内容高度重复。模型虽然记住了这些图片的特征，但它学到的其实是“特定的地板颜

色”或“后面的桌子腿”，而不是坦克真正的几何特征。一旦脱离那个特定的调试房间，模型就无法识别了。

根据上述原因，对系统进行了以下调整与验证。 本人对数据集进行了大清洗和扩充： 1. 本人手动删除了大量背景一模一样、只是坦克位置动了一点点的连拍图片，防止模型过拟合。 2. 本人跑到不同的教室、甚至在室外有阳光和阴影的地方重新拍了些坦克模型照片，补充了俯视、仰视、侧视等各种角度的样本。 3. 在标注的时候，对于一些轻微模糊但人眼还能辨认的图片予以保留，以此增强模型的鲁棒性。通过增加这些多样化的场景，模型在实际联调中的泛化能力显著提高，不会轻易把桌脚认成坦克了。

训练集和验证集划分不合理

在该环节的首次联调中，出现的主要现象如下。 在本地训练 YOLOv5 时，Val（验证集）上的指标非常漂亮，几乎没有丢帧。但是在实机部署测试时，识别率掉得非常惨。

结合命令输出、节点状态和接口工作过程进行检查后，原因可归纳如下。 本人检查了划分训练集和验证集的脚本。原来，本人当时是图省事直接用随机数把整包图片打乱，然后按 8:2 划分的。因为本组采集的数据是连续的视频抽帧，比如视频中第 100 帧和第 101 帧几乎长得一模一样。随机打乱后，第 100 帧分到了训练集，第 101 帧分到了验证集。这样验证集就失去了它原本“检测新数据”的作用，模型直接背答案就能拿高分，但在真实测试的未知场景下立刻原形毕露。

根据上述原因，对系统进行了以下调整与验证。本人重新规划了划分方式：1. 不能单纯靠随机数打乱抽帧。本人把采集的视频段按“场景”和“时间段”分成了几大类。

2. 本人把其中几个独立拍摄的坦克移动视频单独拿出来，作为验证集（Val）；剩下的其他视频抽帧作为训练集（Train）。这样能确保验证集里的坦克背景、光照是训练集里从未出现过的。

3. 重新训练后，虽然验证集上的指标比之前略有下降（降到了88%左右），但这才是模型的真实水平。拿到板端测试时，表现反而比以前稳定多了。

标注框不规范

在该环节的首次联调中，出现的主要现象如下。训练完的模型在检测目标时，画出来的框非常不准。有时候框拉得巨大，把周围的大片地板都包进去了；有时候又只框住了坦克的一个炮塔；更有甚者，坦克明明转了个身，检测框就直接认不出来了。

结合命令输出、节点状态和接口工作过程进行检查后，原因可归纳如下。本组几个人分工标注图片时，没有统一标准。有的人图省事，用 `labelImg` 画框时画得特别粗糙，把目标周围的大量背景都圈了进去；有的人则是对被遮挡的坦克进行“脑补”标注，把没拍到的部分也圈进去了。这种不规范的标注会让模型在学习边界特征时产生混乱，分不清坦克的真正边缘在哪里。

根据上述原因，对系统进行了以下调整与验证。本人拉着组员一起定下了严格的标注规范：

1. 标注框必须紧贴坦克的边缘，边缘空隙越小越好，不允许把不相关的地板、背景包含太多进来。

2. 如果坦克被书本或障碍物挡住了一半，只标注露出来的可

见部分，绝对不要去脑补不可见的部分。 3. 标注完成后，本人用 Python 写了个小脚本，把标注图一张张渲染出来反向检查，发现不合格的全部打回重新画。虽然这一步花了不少时间，但磨刀不误铲雪工，最后训练出的定位精度大幅提升。

图片与标签文件没有一一对应

在该环节的首次联调中，出现的主要现象如下。在终端里运行 `python train.py` 刚刚准备开始训练，进度条还没出来，就直接抛出异常闪退了，提示 `No labels found...`，或者报错说找不到某张图片对应的 `.txt` 文件。

结合命令输出、节点状态和接口工作过程进行检查后，原因可归纳如下。YOLOv5 的数据集格式要求非常严格，`images` 文件夹下的每一张图片，都必须在 `labels` 文件夹下有一个同名且后缀为 `.txt` 的标注文件。本组当时在用 `labelImg` 导出或者拷数据时，可能手动删除了部分不好看的图片，但忘记去删对应的 `txt` 标签文件；或者在复制数据集的时候，漏拷了某个文件夹，导致两个目录里的文件对不上。

根据上述原因，对系统进行了以下调整与验证。 1. 本人检查了 YOLO 数据集的目录结构，确保它的层级符合规范（有 `images/train`, `images/val`, `labels/train`, `labels/val`）。 2. 为了快速排查，本人写了一个简单的 Python 脚本，遍历 `images` 目录下的所有文件名，去 `labels` 目录里比对是否存在同名的 `.txt` 文件。如果多余或者缺失，脚本会自动列出来。 3. 用脚本把多余的无标注图片或孤儿标签清理干净后，再次运行训练命令，YOLO 终于能顺利读取数据集并开始迭代了。

4.7 YOLO 模型训练实现

本项目采用 YOLOv5s 作为基础模型。YOLOv5 将输入图像经过主干网络提取多尺度特征，再由颈部网络融合不同分辨率的信息，最后在多个尺度的检测头上预测目标框、目标置信度和类别概率。与两阶段检测器相比，YOLOv5 能够在一次前向传播中完成候选框生成和分类，适合需要实时处理的机载平台。

训练使用 yolov5s.pt 预训练权重进行迁移学习，输入尺寸为 640，批大小为 16，优化器记录为 SGD。exp5 的训练命令如下：

```
python train.py \
--data data/ccsszz.yaml \
--weights yolov5s.pt \
--epochs 300 \
--batch-size 16 \
--imgsz 640 \
--optimizer SGD
```

训练过程的总损失由框回归损失、目标置信度损失和分类损失构成：

$$L_{total} = \lambda_{box} L_{box} + \lambda_{obj} L_{obj} + \lambda_{cls} L_{cls} \quad (4.4)$$

本任务只有一个类别，因此分类损失接近零属于正常现象。训练期间重点观察训练集与验证集损失是否共同下降，以及 Precision、Recall、mAP 是否进入稳定区间。若训练损失继续下降而验证损失明显上升，则可能出现过拟合；若训练初期即出现标签或路径警告，则应停止训练并修正数据，而不是等待全部轮次结束。

训练环境与训练过程调试

Python、PyTorch 和 CUDA 版本不兼容

在该环节的首次联调中，出现的主要现象如下。在电脑上配置 YOLO 训练环境时，一运行训练就报错 `AssertionError: User-specified CUDA device does not exist`。或者在执行 `import torch; print(torch.cuda.is_available())` 时，返回的居然是 `False`，明明本人电脑里塞了一块英伟达显卡，却只能用慢得像蜗牛一样的 CPU 训练。

结合命令输出、节点状态和接口工作过程进行检查后，原因可归纳如下。这是一个非常经典的配置深问题。本人的 Conda 环境里 PyTorch、CUDA Toolkit 跟本人电脑主机的 NVIDIA 显卡驱动版本对不上。有些同学在配环境时，直接用 `pip install torch` 裸装，这样装下来的默认是 CPU 版本的 torch，或者自带的 CUDA 版本过高，超出了显卡驱动的支持范围。

根据上述原因，对系统进行了以下调整与验证。本人决定推倒重来，彻底清理旧环境：1. 首先在命令行敲 `nvidia-smi`，查看本人显卡驱动所能支持的最高 CUDA 版本（比如本人的是 12.1）。2. 用 conda 创建一个干净的 Python 3.8 虚拟环境：

```
conda create -n yolo_env python=3.8
```

登录 PyTorch 官网，根据本人的显卡驱动版本，找到对应的官方安装指令。本人选择安装支持 CUDA 11.8 的 PyTorch 版本：

```
conda install pytorch torchvision torchaudio pytorch-cuda=11.8 -c pytorch -c nvidia
```

安装好后，在 python 交互界面测试 `torch.cuda.is_available()`，看到返回 `True` 之后才松了一口气。然后再用 `pip install -r requirements.txt` 把其他的 YOLO 依赖装齐，成功用上 GPU 进行加速训练。

数据集 YAML 配置错误

在该环节的首次联调中，出现的主要现象如下。修改好自己的 YAML 配置文件，运行 `train.py --data my_data.yaml` 后，程序直接报错抛出 `KeyError: 'train'` 或者是提示 `Dataset not found`，找不到图片路径。

结合命令输出、节点状态和接口工作过程进行检查后，原因可归纳如下。YAML 文件的格式非常讲究空格和层级。本人之前写路径时，可能直接把 Windows 风格的反斜杠 `\` 拷过来了，或者把相对路径写错了（比如 YOLO 是以项目的根目录为基准寻找路径的，如果写成了以 YAML 文件所在目录为基准，就会找不到文件）。另外，如果只识别“坦克”这一类目标，本人的 `nc`（类别数量）写了 1，但是 `names` 列表里却留了以前的多个类别名字，导致数量和名称对不上。

根据上述原因，对系统进行了以下调整与验证。1. 本人打开 YAML 配置文件，仔细检查了每一个路径。为了避免相对路径的混乱，本人直接把 `train` 和 `val` 指向的图片路径都改成了绝对路径，例如 `/home/user/yolov5/data/images/train/`。2. 核对类别信息，把 `nc: 1` 和 `names: ['tank']` 对应好，保证没有任何多余的类别。3. 检查缩进，

确保所有的 key 后面都有一个空格（比如 nc: 1 中冒号后面必须有空格）。修改后重新运行，数据集成功被解析。

训练时显存不足

在该环节的首次联调中，出现的主要现象如下。刚开始跑训练第一轮（Epoch 0），控制台就突然卡住，然后喷出一长串红色的报错信息：RuntimeError: CUDA out of memory. Tried to allocate..., 训练直接中断退出。

结合命令输出、节点状态和接口工作过程进行检查后，原因可归纳如下。这是因为本人把 YOLO 的训练参数设置得太高了，超出了本人显卡显存的承受极限。比如本人默认设置了 batch-size=32，同时输入图片尺寸 imsz=640，模型又选了比较大的类型，这导致显卡在做前向和反向传播计算时，显存直接被塞爆了。

根据上述原因，对系统进行了以下调整与验证。 1. 最快且最管用的办法就是减小 batch-size。本人把命令里的 --batch-size 32 降到了 16。如果还是报错，本人就继续降到 8。虽然单次处理的样本变少了，但至少能跑起来。 2. 另外，本人还可以把图片的分辨率调小，比如 --imsz 480。不过考虑到这可能会降低对小目标的识别精度，本人优先只降低 batch-size。 3. 在训练时，关闭电脑上其他占用显存的软件（比如浏览器、3D 软件等），把显存全部腾给 YOLO 训练。

模型出现过拟合

在该环节的首次联调中，出现的主要现象如下。训练到了 100 多轮的时候，本

人看到控制台输出的 Train Loss（训练集损失）一直在非常漂亮地往下降，但是 Val Loss（验证集损失）却开始止步不前，甚至还出现了一点反弹。拿实际拍的照片去测，发现模型对训练集里出现过的坦克图片识别得极准，但是稍微换一张稍微偏了点角度的新照片，置信度就暴跌。

结合命令输出、节点状态和接口工作过程进行检查后，原因可归纳如下。这就是典型的“过拟合”。本组的坦克数据集总共只有几百张图，背景相对单调。训练轮数（epochs）设得太多了，模型把本组这几百张图里的每个像素细节、每个噪点都死记硬背了下来，丧失了举一反三的泛化能力。

根据上述原因，对系统进行了以下调整与验证。 1. 本人在训练命令里加入了早停机制（Early Stopping），比如在配置中设置检测到验证集损失连续 20 轮不下降就自动提前终止训练，避免做无意义的过度训练。 2. 本人在训练时开启了 YOLO 内置的数据增强功能（Data Augmentation），比如随机翻转、马赛克拼接、亮度对比度微调等，人工让图片变得多姿多彩。 3. 在最后挑选部署模型时，本人没有使用最后一轮生成的 last.pt，而是使用了在验证集上指标最好的 best.pt 权重文件。

4.8 模型训练结果及对比分析

表 4.3 给出了工作区中三组实际训练记录。exp4 与 exp20 均配置 100 轮，exp5 配置 300 轮并在第 177 轮附近结束。三组训练均采用批大小 16 和 640 像素输入，因此可以直接比较训练轮次变化后的指标。

表 4.3 YOLOv5 实际训练结果对比

训练记录	配置轮数	实际末轮	批大小	输入尺寸	Precision	Recall	mAP@0.5	mAP@0.5:0.95
exp4	100	99	16	640	0.866	0.867	0.818	0.259
exp20	100	99	16	640	0.839	0.840	0.790	0.256
exp5	300	177	16	640	0.958	0.963	0.950	0.411

精确率与召回率分别定义为：

$$P = \frac{TP}{TP + FP}, R = \frac{TP}{TP + FN} \quad (4.5)$$

exp5 的 mAP@0.5 相比 exp4 提高约 0.132，Recall 提高约 0.096，说明增加有效训练轮次后模型在当前验证集上的检测能力明显提升。mAP@0.5:0.95 仍明显低于 mAP@0.5，说明当交并比要求提高时，检测框定位精细度下降，目标尺度、拍摄角度、标注一致性和数据多样性仍是限制因素。

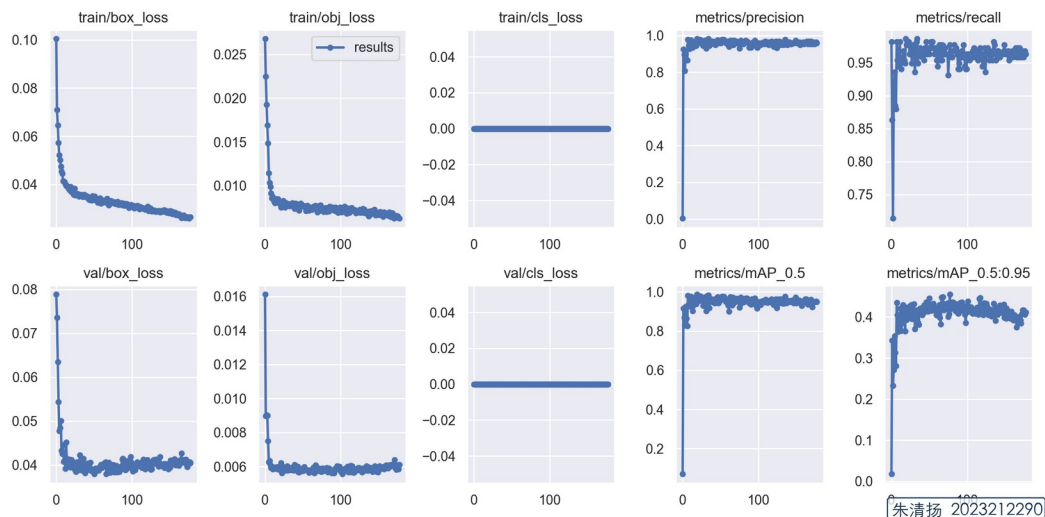


图 4.10 exp5 训练与验证曲线

训练曲线显示框损失和目标损失在前期快速下降，Precision、Recall 与 mAP 同步上升，随后进入波动收敛阶段。PR 曲线在较大召回范围内保持较高精确率，F1 曲线显示中等置信度阈值附近具有较好的综合表现。部署时不能简单使用最低置信度以

追求召回率，因为投弹任务对误检更加敏感，应结合连续帧确认、地理范围和飞行状态进行二次判断。

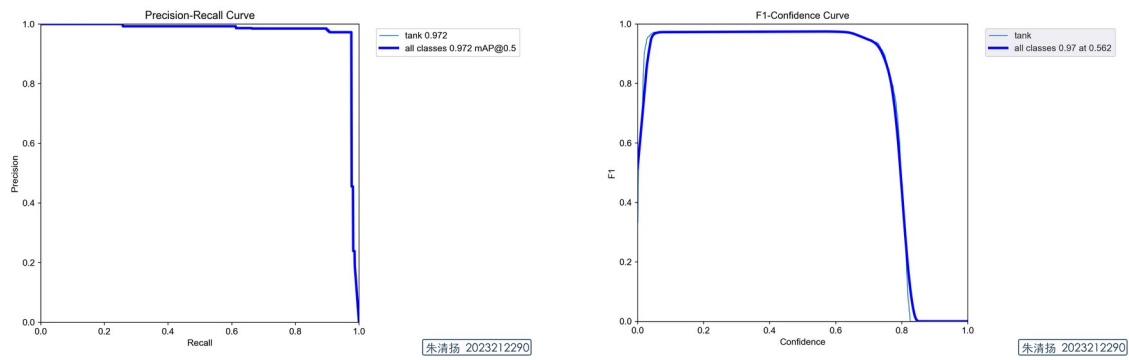


图 4.11 exp5 的 PR 曲线与 F1 曲线

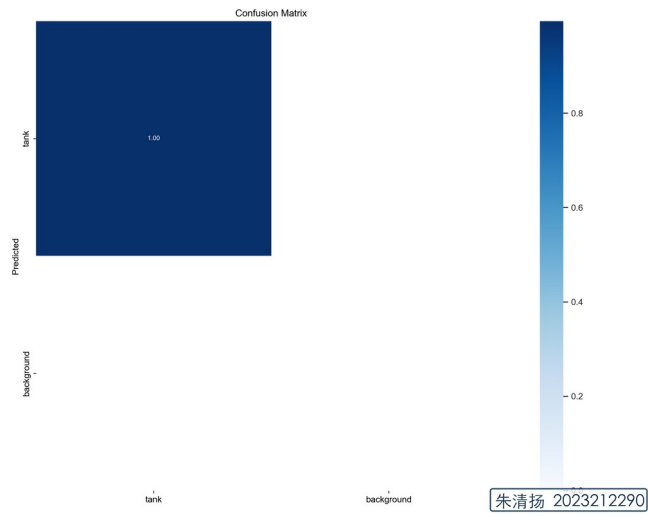


图 4.12 exp5 验证结果混淆矩阵

综合曲线、末轮指标和验证集预测结果，项目选择 exp5 的 best.pt 作为后续模型转换输入。选择 best.pt 而不是 last.pt，是因为前者对应训练过程中验证指标较优的权重，能够降低末期波动对部署结果的影响。

4.9 ONNX 与 RKNN 模型部署

训练得到的 PyTorch 权重不能直接在 RK3566 NPU 上执行，需要先导出为

ONNX 计算图，再通过 RKNN Toolkit 完成算子转换、输入预处理配置、量化和芯片适配。部署流程如图 4.13 所示。

模型格式转换与嵌入式部署流程



图 4.13 模型格式转换与嵌入式部署流程

模型转换时保持输入尺寸为 640×640 ，并核对输入张量排列、RGB/BGR 顺序、归一化方式和输出检测头。板端加载模型后先使用固定测试图验证输出，再接入实时摄像头。典型启动顺序为：

```
ls /dev/video*
```

```
v4l2-ctl --list-devices
```

```
roslaunch rknn_ros camera.launch device:=video0
```

```
roslaunch rknn_ros yolov5.launch chip_type:=RK356X
```

```
rostopic hz /rknn_image
```

转换成功只说明模型文件能够生成，不代表板端结果一定正确。若出现检测框位置错误、类别异常或置信度明显偏低，需要分别检查预处理通道顺序、量化样本、输

出节点、Anchor 和后处理尺度。通过 PC 仿真输出与板端输出对比，可以区分模型转换问题和摄像头问题。

模型转换与板端推理调试

模型规模过大，无法在泰山派上稳定运行

在该环节的首次联调中，出现的主要现象如下。本人在电脑上训练了一个 YOLOv5m 甚至 YOLOv5l 级别的模型，在电脑上测延迟极低。但是当本人把它转换好放到泰山派板子上跑的时候，推理一帧画面居然要花 200 多毫秒（帧率只有 4、5 帧），板子热得烫手，ROS 节点也跟着卡顿，甚至因为内存爆掉直接被系统杀掉进程（OOM Out of memory）。

结合命令输出、节点状态和接口工作过程进行检查后，原因可归纳如下。泰山派开发板使用的是瑞芯微 RK3566 芯片，虽然它带有一个 NPU（神经网络处理器），但整体的算力和内存（通常只有几 G）跟本组电脑的独立显卡根本没法比。YOLOv5m/l 模型的参数量太大，计算量超出了 RK3566 的处理能力，仅靠板子的小身板根本带不动。

根据上述原因，对系统进行了以下调整与验证。

1. 本组必须在模型精度和板端运行速度之间做一个妥协。本人放弃了中大型模型，退回到了最轻量级的 YOLOv5s 模型。
2. 在导出为 ONNX 和 RKNN 时，充分利用瑞芯微官方提供的 NPU 硬件加速。
3. 通过把模型降级为 YOLOv5s，部署到板端后，单帧推理时间成功缩短到了 30~40

毫秒左右，帧率稳定在 25 帧以上，不仅识别速度飞快，而且给其他的 ROS 节点（比如语音、投弹逻辑）留出了充足的 CPU 和内存资源。

best.pt 无法直接转换为 RKNN

在该环节的首次联调中，出现的主要现象如下。训练好 YOLOv5 的 .pt 模型后，本人尝试直接用转换脚本把它转成泰山派 NPU 支持的 .rknn 格式，结果在第一步导出 ONNX 时就报错闪退；或者导出的 ONNX 模型在转换 RKNN 时，提示有算子不支持（Unsupported operators）。

结合命令输出、节点状态和接口工作过程进行检查后，原因可归纳如下。泰山派上的 NPU 硬件对神经网络的算子支持是有限的。原生的 PyTorch 权重文件（.pt）里包含很多复杂的动态操作（比如 YOLOv5 输出端的 Detect 层，里面有很多坐标网格的计算和 Focus 层），这些算子如果直接导出为 ONNX，RKNN 转换工具（RKNN-Toolkit2）是无法识别和量化的。因此，在导出 ONNX 时，必须对 YOLOv5 的网络结构做一些调整，把一些不支持的后处理算子剥离出去，放到 CPU 上运行。

根据上述原因，对系统进行了以下调整与验证。

1. 不能使用官方原版的 export.py 导出。本人使用了瑞芯微专门针对 RKNN 平台优化过的 YOLOv5 导出脚本。该脚本在导出 ONNX 时，会去掉 Detect 层的后处理，只保留网络的主干输出。
2. 在导出命令中，把动态尺寸关闭，固定输入分辨率为 --img-size 640 640。
3. 得到剥离了后处理的 ONNX 模型后，再放到虚拟机的 RKNN-Toolkit2 环境中运行转换脚

本，设置目标平台为 rk3566，开启量化（Quantization），这样就能顺利生成板端可以直接加载的 .rknn 文件了。

RKNN 模型生成成功，但板端检测结果异常

在该环节的首次联调中，出现的主要现象如下。模型转换非常顺利，没有报任何错。但是当本人把它拷到泰山派上运行测试时，屏幕上画出来的检测框到处乱飞，一会儿指着天花板说是坦克，一会儿又把置信度显示成负数，或者对眼前的坦克视而不见。

结合命令输出、节点状态和接口工作过程进行检查后，原因可归纳如下。这多半是因为“数据输入通道”和“后处理逻辑”没有对齐。第一，图像颜色通道不一致。YOLOv5 在训练时默认使用的是 RGB 通道顺序，而泰山派摄像头采集到的数据可能是 BGR 或者 YUV 格式，如果在推理前没有做通道转换（比如把 BGR 转成 RGB），送入模型的数据就是错的。第二，归一化参数不匹配。模型训练时输入像素值被除以了 255（归一化到 0-1 之间），如果在转 RKNN 或者写 C++/Python 推理代码时，没有设置相同的 mean/std 参数，或者重复做了归一化，就会导致模型输入数值错乱。第三，后处理的 Anchor 大小与本组自己修改后的网络不一致。

根据上述原因，对系统进行了以下调整与验证。1. 本人先在虚拟机上运行转换脚本的仿真测试（Simulator），把一张测试图片送进去，看看它输出的 result.jpg 是否正常。如果虚拟机里输出就错，说明是转换配置（RGB/BGR 顺序、量化参数）写错了。

2. 本人检查了推理程序中的预处理部分，确保图像在送入 NPU 前，执行了通道转换

(从 BGR 转为 RGB)，且归一化参数与训练时完全一致。3. 仔细核查了推理代码里的后处理部分，把 anchor 的尺寸参数跟本组 YOLOv5 训练代码里的 yolov5s.yaml 进行了逐一校对，确保没有写错。调整正确后，板端检测结果终于和电脑上一样精准了。

4.10 WireGuard 三端网络搭建

泰山派使用 4G 网络接入公网，地面虚拟机通常位于校园网或局域网，两端均处于 NAT 后，无法依靠私有地址直接建立稳定连接。本项目使用具有固定公网地址的阿里云服务器作为 WireGuard 中心节点，建立 10.0.0.0/24 虚拟网段，拓扑如图 4.14 所示。

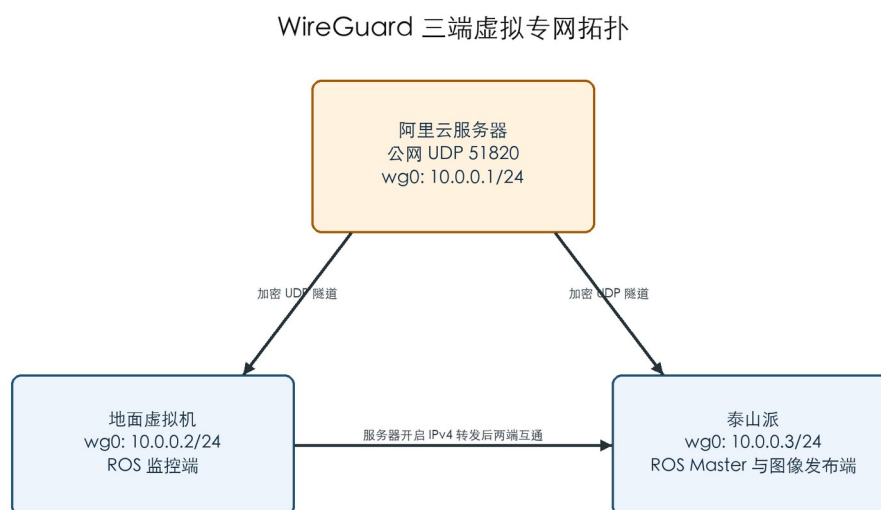


图 4.14 WireGuard 三端虚拟专网拓扑

表 4.4 WireGuard 地址规划

节点	VPN 地址	角色	关键配置
阿里云服务器	10.0.0.1/24	中心节点与转发器	UDP 51820、IPv4 转发
地面虚拟机	10.0.0.2/24	ROS 地面监控端	AllowedIPs 10.0.0.0/24
泰山派	10.0.0.3/24	ROS Master 与图像发布端	4G 接入、Keepalive 25 s

服务器端配置两个 Peer，每个 Peer 只绑定对应客户端的 /32 地址；客户端把 10.0.0.0/24 指向服务器 Peer，使访问任一 VPN 节点的流量进入隧道。服务器配置框架如下：

[Interface]

Address = 10.0.0.1/24

ListenPort = 51820

PrivateKey = <server_private_key>

[Peer]

PublicKey = <vm_public_key>

AllowedIPs = 10.0.0.2/32

[Peer]

PublicKey = <board_public_key>

AllowedIPs = 10.0.0.3/32

服务器还需要开放安全组和本机防火墙的 UDP 51820 端口，启用并持久化 IPv4 转发：

```
sudo sysctl -w net.ipv4.ip_forward=1
```

```
sudo iptables -A FORWARD -i wg0 -o wg0 -j ACCEPT
```

```
sudo iptables -A FORWARD -m conntrack --ctstate RELATED,ESTABLISHED -j ACCEPT
```

```
sudo netfilter-persistent save
```

```
sudo systemctl enable wg-quick@wg0
```

客户端位于 NAT 后时，在 Peer 中配置 PersistentKeepalive = 25，使客户端每隔 25 秒发送小型数据包维持 UDP 映射。网络验证按照“接口、握手、路由、转发、业务”的顺序进行：

```
ip addr show wg0
```

```
sudo wg
```

```
ip route
```

```
ping 10.0.0.1
```

```
ping 10.0.0.2
```

```
ping 10.0.0.3
```

WireGuard 联调过程与验证

服务器能够访问两个客户端，但客户端之间无法互相访问

在该环节的首次联调中，出现的主要现象如下。 阿里云服务器可以分别 ping 通本人的虚拟机（10.0.0.2）和泰山派（10.0.0.3），说明大家其实都已经连上 VPN 了。但是，当本人想在虚拟机上 ping 泰山派，或者在泰山派上 ping 虚拟机时，数据包就像始终没有收到回包（100% packet loss）。

结合命令输出、节点状态和接口工作过程进行检查后，原因可归纳如下。本人在服务器上检查了 `wg show`，发现两端的连接确实是在线的。本人查了些资料才意识到，虽然它们都和服务器建立了 WireGuard 隧道，但阿里云服务器默认是关闭了 IPv4 的内核转发功能的。而且它的防火墙（iptables）也没有配置允许把进入 `wg0` 的流量再次转发给 `wg0`，这就导致服务器只能自己和它们俩通信，没法作为一个“路由器”来中转它们之间的数据包。

根据上述原因，对系统进行了以下调整与验证。1. 本人登录到阿里云服务器，运行了命令 `cat /proc/sys/net/ipv4/ip_forward`，输出结果是 0，印证了本人的猜想。本人先执行 `sudo sysctl -w net.ipv4.ip_forward=1` 临时把转发打开。为了防止服务器以后重启又失效，本人新建了一个配置文件 `/etc/sysctl.d/99-wg.conf`，在里面写上 `net.ipv4.ip_forward=1`，用 `sudo sysctl --system` 保存并生效。2. 接着，本人手动给 iptables 加上了转发规则，允许 `wg0` 自转发：

```
sudo iptables -A FORWARD -i wg0 -o wg0 -j ACCEPT
```

```
sudo iptables -A FORWARD -m conntrack --ctstate RELATED,ESTABLISHED -j ACCEPT
```

做完这些后，本人在虚拟机上重新 `ping 10.0.0.3`。看到屏幕上开始正常蹦出泰山派的延迟回包，延迟大概在 60ms 左右，两边总算是互通了。

WireGuard 一直没有握手记录

在该环节的首次联调中，出现的主要现象如下。在虚拟机或者泰山派上启动 WireGuard 以后，用 `sudo wg` 查看状态，发现一直没有 latest handshake（最新握手时

间) 这一行, 只有发送字节, 接收字节一直是 0, ping 服务器的隧道地址 10.0.0.1 也是完全不通。

结合命令输出、节点状态和接口工作过程进行检查后, 原因可归纳如下。这代表客户端的握手请求根本没有送达服务器。本人排查了配置文件的密钥和 IP, 都没有错。那重点检查方向就是网络端口被阻断了。因为阿里云的服务器默认有安全组防火墙, 如果没有手动在云平台控制台开放 WireGuard 默认的 51820 UDP 端口, 或者是服务器本机的 ufw 防火墙拦截了该端口, 客户端发送的 UDP 数据包就会直接被丢弃, 导致无法握手。

根据上述原因, 对系统进行了以下调整与验证。1. 首先, 本人登录阿里云控制台, 在服务器安全组的入方向规则里, 新增了一条协议为 UDP, 端口为 51820 的放行规则。2. 其次, 登录服务器终端, 检查并设置本机防火墙, 运行 `sudo ufw allow 51820/udp` 放行该端口。3. 接着, 在虚拟机和泰山派客户端上重启 WireGuard 服务:

```
sudo systemctl restart wg-quick@wg0
```

稍等几秒钟后, 在客户端再次输入 `sudo wg`, 可以看到已经出现了 `latest handshake: 3 seconds ago` 这样的握手记录, 收发字节也有了数据, 说明网络成功连通。

三端 VPN 地址重复或不在同一网段

在该环节的首次联调中, 出现的主要现象如下。启动 WireGuard 倒是没有报错,

但网络表现非常诡异。有时候虚拟机能 ping 通服务器，泰山派就不行；有时候一 ping 服务器就持续丢包，甚至本人从服务器 ping 虚拟机的时候，连接莫名其妙跑到了泰山派上。

结合命令输出、节点状态和接口工作过程进行检查后，原因可归纳如下。本人仔细对比了本组三台设备的配置文件，发现是地址规划写得太乱了。比如，泰山派配置里的 Address 填成了和虚拟机一模一样的 10.0.0.2/24，或者有的地方子网掩码写成了 /32 导致不在同一个虚拟局域网网段里。WireGuard 是根据内网 IP 做路由的（通过配置文件里的 AllowedIPs 和本机 Address 绑定），一旦地址冲突或者掩码不对，服务器的路由表就会混乱，不知道把数据包发给谁，从而导致丢包或错乱。

根据上述原因，对系统进行了以下调整与验证。本人决定对三端的 IP 地址进行统一规范和分配，把配置重新统一规划：

1. 阿里云服务器（作为服务端）的 Address 设为 10.0.0.1/24。
2. 本人的虚拟机（客户端 1）设为 10.0.0.2/24。
3. 泰山派开发板（客户端 2）设为 10.0.0.3/24。
4. 修改对应的配置文件（如 /etc/wireguard/wg0.conf），确保三者的 Address 唯一且掩码都是 /24（即 255.255.255.0 网段）。

修改完成后，分别在三台设备上执行 `sudo wg-quick down wg0` 和 `sudo wg-quick up wg0` 重启网络，之后再进行两两互 ping 测试，网络彻底稳定了。

AllowedIPs 配置错误导致路由没有进入隧道

在该环节的首次联调中，出现的主要现象如下。运行 `sudo wg` 能看到握手信息，

说明物理连接是通的。但是在虚拟机上执行 `ping 10.0.0.3`（泰山派）时，控制台直接提示 `Network is unreachable`（网络不可达），或者流量根本没有走 VPN 接口（没有进入隧道）。

结合命令输出、节点状态和接口工作过程进行检查后，原因可归纳如下。在 WireGuard 中，`AllowedIPs` 这个参数极其重要，它不仅决定了允许哪些 IP 段的数据通过，还会在系统路由表中自动生成相应的路由规则。如果虚拟机配置里的 `[Peer]`（服务器）下面只写了 `AllowedIPs = 10.0.0.1/32`，那么虚拟机就只知道去往服务器的流量要走隧道，而去往泰山派（10.0.0.3）的流量则会走默认网卡（比如校园网或局域网），因为找不到路由所以报错。同样，如果服务器端配置 Peer 时，没有把虚拟机的 IP 范围写对，服务器也无法正确转发。

根据上述原因，对系统进行了以下调整与验证。

1. 在虚拟机和泰山派客户端的 `wg0.conf` 中，把服务器 peer 的 `AllowedIPs` 范围扩大。为了让所有 10.0.0.x 网段的流量都走 VPN 隧道，本人将其改写为：`AllowedIPs = 10.0.0.0/24`。
2. 在阿里云服务器的配置文件中，确保针对虚拟机 Peer 的配置里写有 `AllowedIPs = 10.0.0.2/32`，针对泰山派 Peer 的配置里写有 `AllowedIPs = 10.0.0.3/32`。
3. 重启 WireGuard 后，在虚拟机上运行 `ip route` 检查路由表，确认可以看到类似 `10.0.0.0/24 dev wg0 scope link` 的路由项。这时再尝试互 ping，通信恢复正常。

连接校园网时隧道不稳定

在该环节的首次联调中，出现的主要现象如下。本人在宿舍用手机热点或者普通

宽带连 WireGuard 时，网络特别稳定，延迟也很低。但是一旦把电脑和泰山派接入学校的校园网（不管是网线还是校园网 Wi-Fi），VPN 就会经常连不上，或者刚连上几分钟，握手就断了，ping 起来持续丢包，甚至直接卡死。

结合命令输出、节点状态和接口工作过程进行检查后，原因可归纳如下。校园网的网络环境比普通家用网络复杂得多。它通常有网页实名认证、多重 NAT 转换，并且对 UDP 协议的流量有非常严格的限制（因为 WireGuard 纯走 UDP 传输）。校园网网关在检测到持续的 UDP 大流量时可能会限速或直接切断连接；而且在多重 NAT 下，如果没有数据往来，网关上的 UDP 端口映射很快就会失效，导致外面（服务器）的数据根本发不进来。

根据上述原因，对系统进行了以下调整与验证。1. 首先本人确保电脑和泰山派都通过了校园网的网页登录认证，能够正常打开网页。2. 考虑到校园网的 NAT 会快速回收端口，本人在虚拟机和泰山派的配置文件 wg0.conf 的 [Peer] 块下，都添加了保活配置：

PersistentKeepalive = 25

这样客户端每隔 25 秒就会自动给服务器发送一个极小的探测包，强行让校园网的 NAT 路由器维持这个映射通道不关闭。3. 如果在校园网高峰期干扰实在严重，本人会在联调图传和投弹等核心环节时，临时将设备连接到本人手机开的 4G 热点上，以此绕过校园网的策略限制。

WireGuard 刚启动时正常，空闲一段时间后无法访问

在该环节的首次联调中，出现的主要现象如下。每次刚在泰山派和虚拟机上重启 WireGuard 服务时，两边互 ping 都是通的。但是，只要本人把手头工作停下来，去写一会儿别处的代码，过了大约十几分钟再回来看，两边就又 ping 不通了，非得去客户端重新 restart 服务才能恢复。

结合命令输出、节点状态和接口工作过程进行检查后，原因可归纳如下。泰山派连的是 4G 移动网络，虚拟机也处在局域网内，它们都位于各自运营商的 NAT（网络地址转换）后面。当本组在一段时间内没有网络流量时，运营商的 NAT 设备为了节省端口资源，会自动把本组之前建立的 UDP 映射通道给关闭。这样，由于两端都是内网设备，服务器端就失去了主动发送数据给客户端的通路，导致连接中断。

根据上述原因，对系统进行了以下调整与验证。1. 这个问题也是需要靠“保活”来解决。本人打开了泰山派和虚拟机上的 `/etc/wireguard/wg0.conf` 配置文件，在 [Peer] 配置段落中添加了以下参数：

```
PersistentKeepalive = 25
```

它的作用是让处于 NAT 后的客户端每隔 25 秒自动发送一个网络小包，把这个链路“撑开”，不让运营商的防火墙或者 NAT 关掉它。修改好后，执行 `sudo systemctl restart wg-quick@wg0`。经过测试，即使挂机一整晚，第二天早上连过来依然是通的。

服务器重启后转发规则丢失

在该环节的首次联调中，出现的主要现象如下。前一天调好的虚拟机与泰山派互通，第二天因为服务器重启了，结果两端又通不了了。上服务器排查，发现 IPv4 转发明明是开着的，但虚拟机和泰山派之间就是不转数据。

结合命令输出、节点状态和接口工作过程进行检查后，原因可归纳如下。本人之前是用 `iptables -A FORWARD...` 的命令行手动去添加防火墙转发规则的。但在 Linux 系统里，直接用 `iptables` 命令修改的防火墙规则只保存在内存中，一旦系统重启，这些规则就会被全部清空，导致转发链路再次中断。

根据上述原因，对系统进行了以下调整与验证。1. 本人需要把这些防火墙规则做持久化保存。首先在服务器上重新把之前的两条规则输一遍：

```
sudo iptables -A FORWARD -i wg0 -o wg0 -j ACCEPT
```

```
sudo iptables -A FORWARD -m conntrack --ctstate RELATED,ESTABLISHED -j ACCEPT
```

然后安装专门用于保存 `iptables` 规则的工具：

```
sudo apt install iptables-persistent -y
```

在安装过程中，系统会弹出一个蓝色的配置界面，询问是否保存当前的 IPv4 规则，选择“Yes”。3. 如果以后规则有变动，本人可以手动执行 `sudo netfilter-persistent save` 来更新保存的规则。4. 最后确保 WireGuard 服务随系统自启动：

```
sudo systemctl enable wg-quick@wg0
```

这样即使服务器重启，网络转发功能也能自动恢复。

4.11 ROS 跨公网通信与 4G 图传

ROS Master 负责节点注册和连接发现，但不转发实际图像数据。虚拟机能够执行 `rostopic list` 只说明它能够访问 Master；订阅者获得发布者地址后，还需要直接访问发布者注册的 IP。为保证控制面和数据面都经过 WireGuard，泰山派和虚拟机分别使用 VPN 地址设置 `ROS_IP`。

泰山派配置：

```
export ROS_MASTER_URI=http://10.0.0.3:11311
```

```
export ROS_IP=10.0.0.3
```

虚拟机配置：

```
export ROS_MASTER_URI=http://10.0.0.3:11311
```

```
export ROS_IP=10.0.0.2
```

配置写入 `~/.bashrc` 后，在新终端执行 `source ~/.bashrc`，并使用 `echo $ROS_MASTER_URI` 和 `echo $ROS_IP` 检查。图像链路如图 4.15 所示。

ROS 跨公网图像传输链路



图 4.15 ROS 跨公网图像传输链路

原始 640×480 、三通道、8 位图像的单帧数据量约为：

$$D_{frame} = 640 \times 480 \times 3 \approx 0.88 \text{ MiB} \quad (4.6)$$

若以 15 帧每秒传输，未计协议开销的数据率约为 13.2 MiB/s，明显不适合普通 4G 上行链路。因此实际图传使用 `image_transport` 的 JPEG 或 Theora 压缩话题，并在任务需要与网络负载之间选择分辨率和帧率。地面端先用 `rostopic info` 检查消息类型，再选择对应的 `compressed` 或 `theora` 子话题。

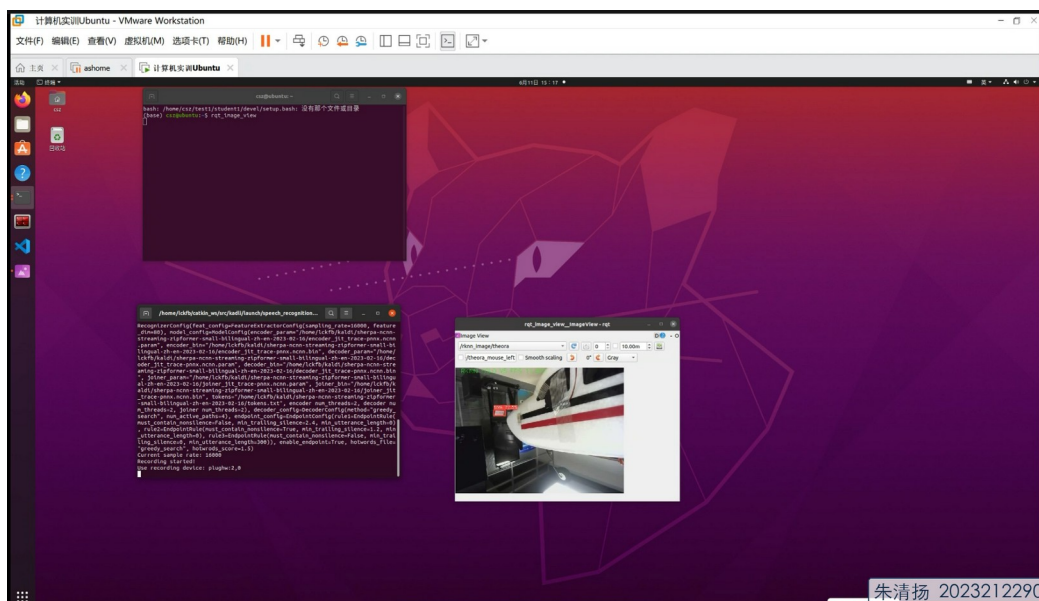


图 4.16 泰山派、WireGuard、ROS 与地面图像显示完整链路

图 4.16 中，虚拟机同时显示远程终端和图像查看窗口，说明 VPN、ROS Master、图像发布和地面解码环节已经贯通。调试时使用 `rostopic hz` 观察实际帧率，并结合 4G 信号、CPU/NPU 温度和网络丢包判断卡顿原因。

ROS 与 4G 图传调试过程

三端可以 ping 通，但虚拟机收不到 ROS 图像

在该环节的首次联调中，出现的主要现象如下。虚拟机和泰山派相互 ping 都没有问题，在虚拟机上跑 `rostopic list` 也能刷出来 `/camera/image_raw` 这样的图像话题名称。但是，当本人打开虚拟机里的 `rqt_image_view` 选择该话题时，窗口里面保持空白，没有任何画面，终端里也没报错，就是干等。

结合命令输出、节点状态和接口工作过程进行检查后，原因可归纳如下。很多同学以为能列出话题就是通了，其实不是。在 ROS 中，ROS Master（本组设在泰山派

上) 只相当于一个“黄页电话簿”。虚拟机去 Master 那里查到有这个图像话题, Master 就会把泰山派的“联系方式”给虚拟机, 让虚拟机自己去跟泰山派建立底层的 TCP/UDP 连接拿数据。如果本组在泰山派的配置文件里, ROS_IP 还是以前物理网卡的 IP (比如校园网 Wi-Fi 的 192.168.x.x), 虚拟机拿到这个 IP 后去连, 但因为在 4G VPN 环境下, 虚拟机根本无法访问泰山派的局域网物理 IP, 所以连接就卡住了, 导致有话题却没图。

根据上述原因, 对系统进行了以下调整与验证。本组必须强制 ROS 在建立底层数据通道时, 全部走 WireGuard 的虚拟 IP (即 10.0.0.x 段)。1. 在泰山派 (10.0.0.3) 的当前终端或 ~/.bashrc 中写入:

```
export ROS_MASTER_URI=http://10.0.0.3:11311
```

```
export ROS_IP=10.0.0.3
```

在虚拟机 (10.0.0.2) 中写入:

```
export ROS_MASTER_URI=http://10.0.0.3:11311
```

```
export ROS_IP=10.0.0.2
```

修改完后, 在两端各自的终端里执行 source ~/.bashrc 刷新环境变量, 然后重新启动泰山派的摄像头节点和虚拟机上的接收端。再次打开 rqt_image_view, 画面就正常传过来了。

新终端中的 ROS 环境变量没有生效

在该环节的首次联调中, 出现的主要现象如下。本人在一个终端里调好了配置,

一切都正常。但是当本人开了一个新终端想去跑别的话题命令时，却提示 `Unable to contact my own XMLRPC server...` 或者干脆找不到 ROS Master。

结合命令输出、节点状态和接口工作过程进行检查后，原因可归纳如下。这是因为本人之前的一些 `export ROS_IP=...` 命令只是在当时那个终端窗口里临时生效的。Linux 每一个新打开的终端都是一个新的 shell 会话，它不会继承其他终端的临时变量。如果没把这些配置写进用户的 shell 配置文件里，每次新建终端环境变量就会全部清空。

根据上述原因，对系统进行了以下调整与验证。本人把这些 ROS 的环境变量配置直接固化到用户的配置文件中： 1. 打开家目录下的配置文件：`nano ~/.bashrc`。 2. 滑动到文件最底部，把配置加进去（以泰山派为例）：

```
export ROS_MASTER_URI=http://10.0.0.3:11311
```

```
export ROS_IP=10.0.0.3
```

保存退出后，在新打开的终端里执行 `source ~/.bashrc`。

为了保险，本人敲了 `echo $ROS_IP` 和 `echo $ROS_MASTER_URI`，确认输出的值和本人的 WireGuard 虚拟 IP 一致。这样以后开再多终端也不用手动配置了。

ROS 节点异常退出后再次启动失败

在该环节的首次联调中，出现的主要现象如下。有时候调试代码出错，或者本人直接用 `Ctrl+C` 强行终止了程序，当本人马上想再次用 `roslaunch` 启动它时，控制台就

会刷出一堆红字，报错说“节点已经存在”、“设备已被占用”或者“端口冲突”，程序直接闪退。

结合命令输出、节点状态和接口工作过程进行检查后，原因可归纳如下。在 ROS 里，一个 launch 文件可能会拉起好多个子进程（比如摄像头驱动、YOLO 识别、语音识别等）。有时候本组按下 Ctrl+C，主进程退出了，但是某些底层的子进程卡在系统里变成了残留进程，继续死死霸占着 USB 摄像头设备（如 /dev/video0）或者网络端口。这时候再次启动，新进程去申请摄像头和端口就会因为被占用而失败。

根据上述原因，对系统进行了以下调整与验证。遇到这种情况，最忌讳的就是不进行状态确认便直接去重启整个泰山派板子，那样太浪费时间。本人通常会按照以下步骤排查和清理：

1. 先用 `rostopic list` 看看还有没有残留的节点名在 Master 注册。如果有，用 `rostopic kill /节点名称` 强行注销它。
2. 如果是硬件设备被霸占，本人会在后台搜进程：

```
ps -ef | grep ros
```

```
ps -ef | grep camera
```

找到那些残留进程的 PID（进程号）后，直接执行 `sudo kill -9 进程号` 强行杀掉。

特别是语音识别模块（sherpa），有时也会留下孤儿进程，也可以用 `ps -ef | grep sherpa` 找出并杀掉。清理干净后，重新启动程序就能一次成功。

摄像头节点提示设备不存在或无法打开

在该环节的首次联调中，出现的主要现象如下。运行课件里的启动命令

roslaunch ... camera.launch device:=video0 时，控制台报错说 VIDIOC_REQBUFS: Device or resource busy 或者 Cannot identify '/dev/video0'，节点直接挂掉。

结合命令输出、节点状态和接口工作过程进行检查后，原因可归纳如下。硬件调试过程中，摄像头可能会因为接线松动被本组重新插拔，或者板子重启过。Linux 系统对于热插拔的 USB 设备，分配的序号是动态的。比如你上次插上是 video0，拔掉再插上可能就被系统分配成了 video1 甚至 video9。如果代码或者启动命令里死板地写着 video0，就根本找不着设备。

根据上述原因，对系统进行了以下调整与验证。1. 每次启动摄像头节点前，本人都会先在终端里执行 `ls /dev/video*`，看看当前系统里到底挂载了哪些摄像头设备。2. 如果不太确定哪个才是本组的 USB 摄像头，本人还会用 `v4l2-ctl --list-devices` 命令（如果没有这个工具，可以用 `sudo apt install v4l-utils` 安装），它会清晰地打印出每个 video 序号对应的硬件名称。3. 确认了当前摄像头的实际设备号（假设是 `/dev/video1`）后，在启动命令里传入正确的参数：

```
roslaunch usb_cam usb_cam-test.launch video_device:=/dev/video1
```

这样就能顺利避开设备号对不上的问题。

图像话题已经发布，但 **rqt_image_view** 仍显示空白

在该环节的首次联调中，出现的主要现象如下。本人在虚拟机上用 `rostopic list` 检查，确实能看到 YOLO 处理后的话题，用 `rostopic hz /rknn_image` 也能看到有十几

赫兹的稳定帧率输出，说明泰山派正在持续往外发图。但是本人在 `rqt_image_view` 窗口里选了这个话题，屏幕还是黑的，或者显示一个灰色的叉。

结合命令输出、节点状态和接口工作过程进行检查后，原因可归纳如下。这通常是因为话题的数据格式和本组的接收端不匹配。比如，泰山派发送的是压缩格式的图像数据（`theora` 或者 `compressed` 格式），而本组在虚拟机 `rqt_image_view` 里默认选择的是普通 `raw` 格式；或者相反，YOLO 推理完之后输出话题的名称有细微差别（比如本组以为是 `/rknn_image`，实际上节点输出的话题是 `/rknn_image/compressed`）。另外，如果网络存在较大的丢包，导致图像的某些关键帧丢失，解码器无法还原画面，也会表现为空白。

根据上述原因，对系统进行了以下调整与验证。1. 本人会先用 `rostopic info` 话题名 仔细看这个话题发布的 `Message` 类型（比如是 `sensor_msgs/Image` 还是 `sensor_msgs/CompressedImage`）。2. 在 `rqt_image_view` 的下拉菜单中，不要只看话题的名字，一定要注意看它后面带不带后缀。如果是压缩图，必须选带 `/compressed` 或 `/theora` 后缀的子选项，这时候 `rqt_image_view` 才会启用对应的插件进行解码。3. 如果依然不显示，本人会在虚拟机终端用以下命令尝试用另一个轻量级的 `viewer` 接收，以此排除是否是 `rqt` 软件本身的卡顿问题：

```
roslaunch image_view image_view image:=/话题名
```

4G 图传延迟较大或画面不连续

在该环节的首次联调中，出现的主要现象如下。虚拟机上终于能看到泰山派传过

来的画面了，但体验不满足实时观察要求。画面卡顿得很厉害，像动画片一样，而且延迟很大，本人用手在泰山派摄像头前晃，虚拟机上要等个 3、4 秒才有反应。

结合命令输出、节点状态和接口工作过程进行检查后，原因可归纳如下。4G 网络（特别是本组实训用的小流量数据卡）的上行带宽非常有限，而且时延不像局域网那样稳定。如果本组直接把摄像头采集到的原始无损图像（Raw Image，一帧可能就有几兆大小）塞进 ROS 话题里通过 4G 传输，带宽随后就会被吃满，导致数据包在网络队列里严重堆积和丢包。此外，如果泰山派没有安装散热风扇，长时间高负荷运行 YOLO 目标识别会导致 CPU/NPU 温度飙升到 80°C 以上，系统触发温度保护主动降频，处理一帧画面变慢，也会造成图传极度卡顿。

根据上述原因，对系统进行了以下调整与验证。

1. 本人没有直接传输大体积的 `/camera/image_raw`，而是使用了 ROS 自带的图像压缩节点 `image_transport`，将图像压缩成 JPEG 格式后再通过 `/camera/image_raw/compressed` 话题发送。这样传输带宽直接缩减了 90% 以上。
2. 在摄像头的 `launch` 文件中，本人把分辨率从原先的 1280x720 降低到了更实用的 640x480，并将帧率限制在 15 帧左右。这对于坦克目标识别来说完全够用了，而且极大地减轻了网络负担。
3. 排查卡顿时，本人还在泰山派上挂载了散热片并用小风扇吹着，用 `sensors` 或以下命令监控温度，保证其处于 60°C 以下的正常工作频段：

```
cat /sys/class/thermal/thermal_zone0/temp
```

经过优化后，图传延迟成功被本人压到了 0.5 秒以内，画面非常顺畅。

4G 数据卡插入后仍然无法联网

在该环节的首次联调中，出现的主要现象如下。本组把 4G 模块接在泰山派上，小灯也在闪，用 `lsusb` 也能看到模块的硬件名称。但是板子就是没网，执行 `ping www.baidu.com` 提示 `Temporary failure in name resolution`。

结合命令输出、节点状态和接口工作过程进行检查后，原因可归纳如下。4G 模块在 Linux 上并不是插上就能直接用的，它需要经过“拨号”获取 IP 地址的过程。本组用的数据卡运营商不同（移动、联通或电信），对应的接入点名称（APN）也完全不一样。如果直接跑课件里默认的拨号脚本，脚本里可能写死了移动的 APN，如果本组插的是联通卡，拨号就会失败。此外，有时候系统虽然生成了虚拟网卡，但没有自动通过 DHCP 获取 IP，或者默认路由依然走的是没有联网的以太网口。

根据上述原因，对系统进行了以下调整与验证。

1. 本人先用 `microcom -s 115200 /dev/ttyUSB2`（或者是对应的 AT 指令串口）连上模块，发送 `AT+CPIN?` 检查 SIM 卡状态是否正常，再发 `AT+CSQ` 看看信号强度是否足够。
2. 确认卡没问题后，本人查看了拨号脚本。本人把脚本中的 APN 参数修改为与本人手头这张数据卡运营商相符的配置（例如联通卡改为 `3gnet`）。
3. 运行拨号脚本（如 `quectel-CM`）后，本人观察输出日志，确认它成功拿到了 `wwan0` 接口的 IP 地址。
4. 拨号成功后，本人用 `ip route` 检查了默认网关，发现路由表里的默认网关还是指向已断开的以太网。于是本人运行以下命令删除了旧的失效路由，并将默认路由指向 `wwan0` 接口：

```
sudo route del default
```

调整后访问公网就随后通了。

SSH 能连接泰山派，但图形程序无法显示

在该环节的首次联调中，出现的主要现象如下。 本人通过虚拟机 SSH 远程连上了泰山派，在终端里敲命令都没问题。但是当本人尝试运行一些带界面的程序（比如启动带有实时显示窗口的 YOLO 识别脚本，或者运行一些可视化调试工具）时，终端就会报错：Error: Can't open display: 或者 cannot connect to X server，程序打不开。

结合命令输出、节点状态和接口工作过程进行检查后，原因可归纳如下。 普通的 SSH 连接只传输纯文本数据。当本组在泰山派上运行带图形化界面的程序时，程序产生的图形窗口需要一个“显示服务器（X Server）”来渲染。因为泰山派上没有接显示器，它就不知道把窗口画到哪里。如果本组没有开启 X11 转发，泰山派就无法把图形渲染数据通过网络发送给虚拟机的显示服务。

根据上述原因，对系统进行了以下调整与验证。 1. 在使用 SSH 连接泰山派时，必须带上 -Y 或 -X 参数，这会启用受信任的 X11 转发功能。连接命令改为：

```
ssh -Y lckfb@10.0.0.3
```

虚拟机端也必须运行着 X11 服务（Ubuntu 虚拟机桌面环境默认就是开启的）。

本人还检查了泰山派上的 /etc/ssh/sshd_config 文件，确保其中 X11Forwarding yes 这一行没有被注释掉。

需要说明的是，因为 4G 组网下网络带宽有限，通过 X11 转发直接传图形界面

的延迟非常大。所以本组在实训时，主要是通过 ROS 话题把摄像头图像发回虚拟机，然后在虚拟机本地用 `rqt_image_view` 渲染显示，这样效率要高得多，不建议直接在远程 SSH 里强行打开图形窗口。

4.12 语音识别与航点推送

语音识别模块将麦克风输入转换为任务关键词，任务节点根据有效口令调用 MAVROS 航点推送和模式切换服务。其工作过程为：启动语音模型，持续读取音频，判断是否识别到预设口令，解析航点文件，构造 `mavros_msgs/Waypoint` 数组，调用 `/mavros/mission/push` 写入飞控，随后根据任务阶段切换飞行模式。

航点推送函数需要检查服务调用是否成功以及飞控接受的航点数量，典型逻辑如下：

```
if (waypoint_push_client.call(wp_srv) && wp_srv.response.success) {  
  
    ROS_INFO("Waypoints pushed successfully");  
  
} else {  
  
    ROS_ERROR("Failed to push waypoints");  
  
}
```

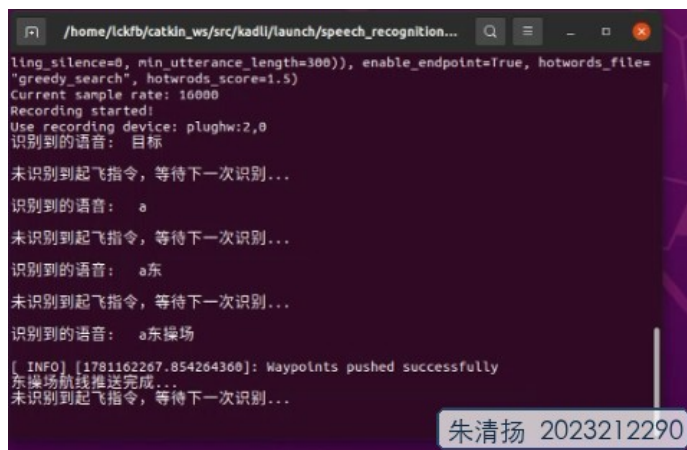
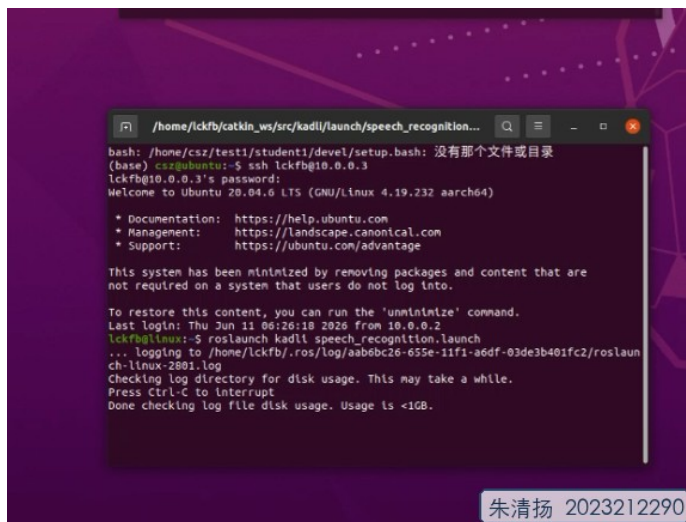


图 4.17 语音识别节点运行及航点推送结果

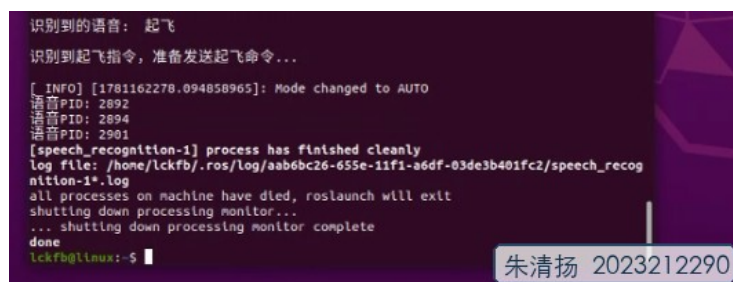


图 4.18 识别起飞指令后任务模式切换

终端输出中出现 Waypoints pushed successfully 和 AUTO 模式切换信息，说明语音识别结果已经进入航点任务链，而不是停留在文本输出层面。语音模块与图传、目标识别和投弹程序共用 ROS 环境，因此网络地址和 Master 配置必须一致。

4.13 目标检测与地理坐标解算

目标定位需要完成图像坐标、相机坐标、机体坐标、北东坐标和地理坐标之间的转换，关系如图 4.19 所示。设检测框中心为 (u, v) ，相机主点为 (c_x, c_y) ，相机等效焦距为 f_x, f_y ，相对高度为 h ，则在相机近似垂直向下且地面局部平坦时，可将像素偏移投影为地面位移：

$$x_c = \frac{u - c_x}{f_x} h, y_c = \frac{c_y - v}{f_y} h \quad (4.7)$$

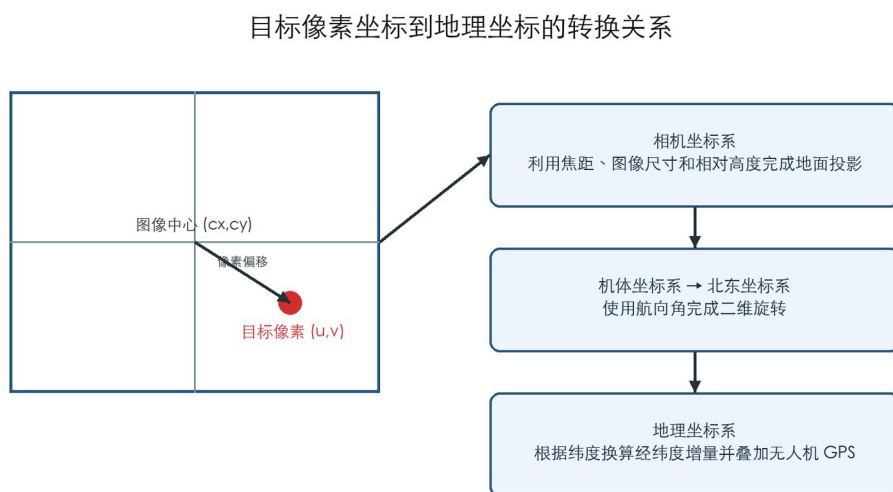


图 4.19 目标像素坐标到地理坐标的转换关系

若无人机航向角为 ψ ，将机体系位移旋转到东、北方向：

$$\begin{bmatrix} x_e \\ y_n \end{bmatrix} = \begin{bmatrix} \cos \psi & -\sin \psi \\ \sin \psi & \cos \psi \end{bmatrix} \begin{bmatrix} x_c \\ y_c \end{bmatrix} \quad (4.8)$$

在局部小范围内，地球半径取 R_e ，无人机纬度为 φ ，经纬度增量近似为：

$$\Delta lat = \frac{y_n}{R_e} \frac{180}{\pi}, \Delta lon = \frac{x_e}{R_e \cos \varphi} \frac{180}{\pi} \quad (4.9)$$

为避免单帧检测框抖动直接造成目标经纬度跳变，程序先检查坐标是否位于任务区域，再把有效样本加入时间窗口。三秒内样本数量达到阈值后计算平均值并更新炮弹目标：

```
void Target::final_target_coordinates(double target_lat, double target_lon) {  
  
    if (target_lat < SOUTH_lat_Scope || target_lat > NORTH_lat_Scope ||  
  
        target_lon < WEST_Lon_Scope || target_lon > EAST_Lon_Scope) {  
  
        return;  
  
    }  
  
}
```

```
Final_Coordinates sample;
```

```
sample.timestamp = ros::Time::now();
```

```
sample.lat = target_lat;
```

```
sample.lon = target_lon;
```

```
if (final_coordinates_buf.empty() ||
```

```
    sample.timestamp - final_coordinates_buf.front().timestamp <
```

```

        ros::Duration(3.0) {

final_coordinates_buf.push_back(sample);

return;

}

if (final_coordinates_buf.size() > 20) {

double sum_lat = 0.0;

double sum_lon = 0.0;

for (const auto &item : final_coordinates_buf) {

sum_lat += item.lat;

sum_lon += item.lon;

}

drop_coordinates_lat = sum_lat / final_coordinates_buf.size();

drop_coordinates_lon = sum_lon / final_coordinates_buf.size();

find_goal = 1;

}

final_coordinates_buf.clear();

}

```

目标坐标解算调试

识别框中心坐标波动，导致目标经纬度不稳定

在该环节的首次联调中，出现的主要现象如下。 当本人的 YOLO 模型在持续识

别地面上的坦克时，由于光照变化或目标轻微晃动，识别框在画面里会细微地抖动（比如中心坐标一直在几十个像素范围内来回跳）。这直接导致根据这个坐标算出来的坦克实际经纬度在地图上持续漂移，投弹程序根本无法锁定一个稳定的位置。

结合命令输出、节点状态和接口工作过程进行检查后，原因可归纳如下。YOLO 每一帧输出的检测框都是独立计算的，会受到图像噪点、阴影和置信度波动的干扰。如果本组直接把单帧算出来的像素坐标拿去计算经纬度，这种微小的“帧间抖动”就会被飞行高度和相机的投影矩阵无限放大，反映在地理坐标上就是十几米的漂移。

根据上述原因，对系统进行了以下调整与验证。本人在自动投弹的 ROS 节点里加入了一个“多帧滑动平均和滤波”的缓存机制：1. 本人没有直接用单帧的坐标去算经纬度，而是建立了一个容量为 10 帧的队列，把连续识别到的坦克中心像素坐标存进去。2. 如果新一帧的坐标偏离上一帧太远（比如判定为误检抖动），本人就直接丢弃这帧数据。3. 对队列里的坐标求平均值，作为平滑后的目标中心点，再去换算地理坐标。通过引入这个简单的滑动窗口滤波，算出来的目标经纬度终于不再持续乱飘，定位稳定在了一个很小的半径内。

像素坐标转换为地理坐标时误差较大

在该环节的首次联调中，出现的主要现象如下。模型能很准地把坦克框出来，但是计算出来的坦克经纬度，在地图上跟坦克的实际位置能偏出去好几米甚至十几米，误差大得根本没法用来指导投弹。

结合命令输出、节点状态和接口工作过程进行检查后，原因可归纳如下。像素坐标转地理坐标（经纬度）是一个复杂的几何投影过程，它需要融合好几个传感器的数据。本人检查了计算公式，发现这中间涉及的变量太多了：无人机当前的 GPS 位置、飞控回传的相对高度、相机的视场角（FOV）、无人机的偏航角（Yaw，即机头朝向），还有俯仰角（Pitch）和横滚角（Roll）。本人之前在写转换代码时，忽略了相机焦距和像素比例的换算，而且没有把无人机的实时姿态角（尤其是航向角）考虑进去，导致一旦飞机稍微转个弯，投影方向就彻底偏了。

根据上述原因，对系统进行了以下调整与验证。本人把整个坐标转换逻辑拆成了三个阶段，逐步写代码验证： 1. 第一步：先不考虑高度，利用相机内参（焦距、中心点），把图像上的像素坐标 (u, v) 转换成相机坐标系下的物理坐标。 2. 第二步：结合飞控读取到的实时高度和云台/相机安装角度，投影到以无人机中心为原点的地面物理坐标系下 (X_drone, Y_drone) 。 3. 第三步：这一步最关键，必须获取飞控的实时航向角（Yaw），把物理坐标绕 Z 轴旋转，对齐到“北-东-地”的地理坐标系下，然后再结合无人机当前的 GPS 坐标（经纬度），换算出目标的实际经纬度。 4. 本人在代码里把角度和弧度转换仔细校对了一遍，修正了方向定义。重新联调后，定位误差被缩减到了 1 米左右，基本满足了投弹要求。

投弹程序中的单位不统一

在该环节的首次联调中，出现的主要现象如下。程序运行起来了，但是算出来的“自由落体时间”或者“投弹延迟”特别离奇。比如，算出来炸弹要掉 100 多秒，或者延

迟时间是个负数，导致程序直接在错误的时间点触发投弹。

结合命令输出、节点状态和接口工作过程进行检查后，原因可归纳如下。这是因为 ROS 消息和飞控回传的各种物理量单位不统一，而本人在写计算公式时直接把它们混在一起算了。例如，飞控回传的高度通常是以“米（m）”为单位，但是 GPS 相对高度在某些 ROS 节点里可能是以“毫米（mm）”或者“厘米（cm）”发布的；无人机当前的水平移动速度有的是“米每秒（m/s）”，有的则是“厘米每秒（cm/s）”。如果直接用 高度 ÷ 速度，单位不对，算出来的时间自然差了十万八千里。

根据上述原因，对系统进行了以下调整与验证。1. 本人重新梳理了投弹节点订阅的所有 ROS 话题（比如 /mavros/global_position/local 和 /mavros/local_position/velocity_local），去源码里逐个确认它们的数据单位。2. 在投弹算法的输入端，本人写了一段数据规范化逻辑，把所有的高度、距离统一强制转换为标准单位“米（m）”，速度全部转换为“米每秒（m/s）”。3. 在计算公式的每一步都打印出调试日志，输出高度、当前速度、自由落体时间等，以此来对比数值是否合乎常理。单位统一后，计算结果终于恢复正常。

4.14 自动投弹模型与程序实现

投弹模型采用固定翼在短时间内水平匀速、弹体释放后竖直方向做自由落体的近似。设相对高度为 h 、重力加速度为 g ，理论下落时间为：

$$t_f = \sqrt{\frac{2h}{g}} \quad (4.10)$$

若水平速度为 v ，理论提前距离为：

$$d_f = v t_f \quad (4.11)$$

设无人机到目标的当前水平距离为 d ，通信、程序处理和舵机动作补偿为 t_c ，则等待时间为：

$$t_d = \frac{d}{v} - t_f - t_c \quad (4.12)$$

程序判断流程如图 4.20 所示。进入核心计算前必须确认目标已锁定、水平速度大于阈值、高度为正、位置数据有效且坐标位于任务范围内。若等待时间尚大于释放窗口，程序继续更新飞行状态；进入窗口后执行 `open_boom()`，并设置锁定标志防止重复触发。

自动投弹判断程序流程

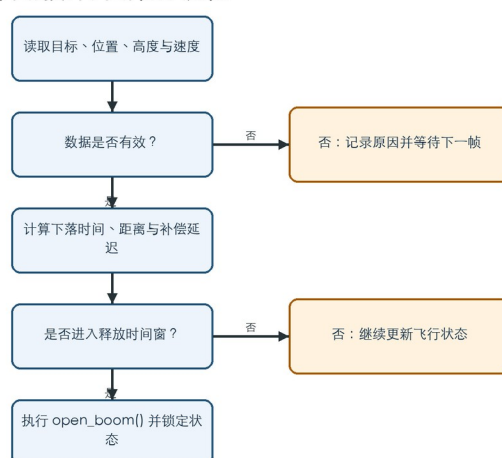


图 4.20 自动投弹判断程序流程

```

void fire() {

    const double velocity =

        std::hypot(drop_boom.velocity_x, drop_boom.velocity_y);

    if (!find_goal || velocity < 0.2 ||

```

```

    uav_coordinates.rel_alt <= 0.0) {

    ROS_WARN_THROTTLE(1.0, "Drop condition invalid.");

    return;

}

const double distance = distance_calculate(

    uav_coordinates.lat, uav_coordinates.lon);

const double fall_time =

    std::sqrt(2.0 * uav_coordinates.rel_alt / 9.8);

const double actuator_compensation = 0.12;

const double delay =

    distance / velocity - fall_time - actuator_compensation;

ROS_INFO("velocity=%.3f m/s, distance=%.3f m, delay=%.3f s",

    velocity, distance, delay);

if (delay <= 0.0) {

    open_boom();

} else if (delay <= 0.3) {

    ros::Duration(delay).sleep();

    open_boom();

```

```
}  
  
}
```

程序将距离统一为米、速度统一为米每秒、角度统一在输入处转换为弧度。0.2 m/s 的速度门限用于避免地面静态阶段或异常速度造成除零。0.12 s 补偿用于覆盖通信、调度和舵机机械动作延迟。真实飞行还受到风、姿态变化和空气阻力影响，因此系统通过连续帧确认、任务区域约束和地面分级测试降低误触发风险。

4.15 PWM 舵机控制与机械释放

常用舵机使用 20 ms 周期、50 Hz 的 PWM 信号，高电平脉宽决定转角。1.0 ms、1.5 ms 和 2.0 ms 脉宽通常分别对应舵机一端、中位和另一端，实际释放角度需根据舵机与挂钩结构标定。PWM 控制原理如图 4.21 所示。



图 4.21 舵机 PWM 周期与脉宽控制示意

占空比定义为高电平时间与周期之比：

$$\eta = \frac{t_{high}}{T} \times 100\% \quad (4.13)$$

当周期为 20 ms 时, 1.0 ms、1.5 ms 和 2.0 ms 脉宽分别对应约 5%、7.5% 和 10% 的占空比。Linux sysfs PWM 接口必须按照导出通道、设置周期、设置极性、设置占空比和使能输出的顺序操作:

```
int PWM_Init() {

    write_value(PWM_EXPORT_PATH, "0");

    write_value(PWM_PERIOD_PATH, "20000000");

    write_value(PWM_POLARITY_PATH, "normal");

    write_value(PWM_DUTY_PATH, "1500000");

    write_value(PWM_ENABLE_PATH, "1");

    return 0;

}

int set_PWM(int duty_cycle_ns) {

    return write_value(PWM_DUTY_PATH,

                      std::to_string(duty_cycle_ns));

}

void open_boom() {

    set_PWM(2000000);

}
```

```
void close_boom() {  
  
    set_PWM(1000000);  
  
}
```

调试时先运行独立 PWM 程序，确认权限、通道、周期和舵机动作，再接入投弹节点。若舵机已经转动但弹体未释放，需要检查脉宽范围、舵机扭矩、独立供电和机械摩擦，而不能仅依据程序日志判断成功。

4.16 地面分级测试

自动投弹同时涉及飞行、视觉、网络、算法和机械机构，直接实飞难以定位问题且风险较高。本项目采用四级测试逐步扩大验证范围，如图 4.22 所示。

系统分级测试与风险收敛过程

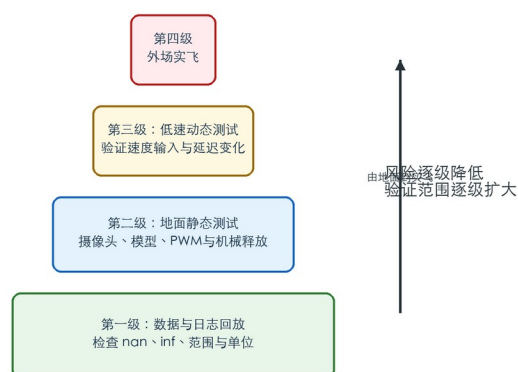


图 4.22 系统分级测试与风险收敛过程

表 4.5 分级测试内容及通过条件

阶段	输入与环境	主要观察量	通过条件
数据与日志回放	保存图像、飞行状态或模拟输入	坐标、速度、高度、延迟	无 nan、inf 和越界触发
地面静态测试	固定机体、目标图、舵机挂载	检测框、PWM、机械释放	目标稳定识别，舵机可靠动作
低速动态测试	手持机体移动	速度门限、延迟变化	数值随距离和速度合理变化
外场实飞	完整飞行与网络环境	起飞、图传、任务状态	飞行稳定，地面端有实时画面

投弹时机计算的数值处理

飞行速度接近零时延迟计算异常

在该环节的首次联调中，出现的主要现象如下。本组在地面做静态调试，或者无人机在空中悬停（速度极低）的时候，投弹程序会突然报错崩溃，或者打印出来的投弹延迟数值直接变成了 inf（无穷大）或 nan（非数）。

结合命令输出、节点状态和接口工作过程进行检查后，原因可归纳如下。投弹算法中有一步是计算从当前位置到释放点的水平投掷距离。公式里需要计算“提前量”，这里面涉及一个除法：用“水平距离除以当前水平速度”。当无人机在地面静止不动，或者空中悬停时，它的水平速度接近于 0。在代码里，直接用一个数去除以接近 0 的速度，就会发生计算机里经典的“除零错误”，导致结果溢出或崩溃。

根据上述原因，对系统进行了以下调整与验证。本人在投弹计算逻辑的入口处，加了一个严格的状态和数值阈值判断：1. 读取到速度后，先取水平速度的模长（即计算 $v = \sqrt{v_x^2 + v_y^2}$ ）。2. 判断如果速度低于 0.2 米每秒（认为处于静止或悬停状态），则直接跳过投弹延迟计算，不执行除法，同时在控制台打印提示。3. 只有当飞机速度大于该阈值，且其他定位和目标数据都有效时，才允许进入核心计算。这样就彻底杜绝了除零带来的程序异常。

理论投弹模型没有考虑全部外界因素

在该环节的首次联调中，出现的主要现象如下。 算法公式在仿真里跑得很完美，但是本组在室外进行挂载投放测试时，炸弹总是砸不准坦克。有时候会往风向的下游偏出去很多，或者因为舵机反应慢，导致投放点总是滞后。

结合命令输出、节点状态和接口工作过程进行检查后，原因可归纳如下。 本组最开始写的公式是一个非常理想化的“物理真空模型”：只根据公式算出自由落体时间，然后乘以无人机的水平速度得出水平提前量。但现实中，炸弹在下落过程中会受到空气阻力和侧向风的吹袭；而且从本组的程序发出“投放”指令，到板子串口发信号、舵机转动、挂钩脱开，这中间存在几十到上百毫秒的“系统物理延迟”。这些外界干扰和延迟在高速飞行下，都会变成好几米的物理偏差。

根据上述原因，对系统进行了以下调整与验证。 考虑到本组在短时间内很难建出一个完美的空气动力学模型，本人采用了“物理公式 + 经验补偿系数”的工程解决办法：

1. 本人在投弹延迟公式中，加入了一个手动标定的补偿时间常数，用来抵消串口通信和舵机动作的物理延迟（经过本组在地面用秒表和录像反复测试，这个物理延迟大约在 120 毫秒左右）。
2. 针对风阻，本人引入了一个简单的阻力修正系数。
3. 在报告中，本人诚实地说明了目前采用的仍是基于水平匀速假设的简化投弹模型，主要用于原理性验证。如果以后要提高精度，需要加入飞控回传的风估算数据（Wind Estimation）以及更高级的弹道补偿算法。

PWM 与机械释放调试

PWM 初始化或占空比设置失败

在该环节的首次联调中，出现的主要现象如下。调试时，目标识别和距离计算都工作正常，当程序判定到达投弹点并输出 `open_boom()` 日志时，接在板子上的舵机却始终没有产生预期动作。本人去看控制台，发现上面弹出一行报错：`Failed to open pwm export file 或者 Permission denied`。

结合命令输出、节点状态和接口工作过程进行检查后，原因可归纳如下。泰山派是通过 Linux 底层的 `/sys/class/pwm/` 接口来输出 PWM 信号控制舵机的。舵机不动有两方面原因：第一是权限问题。系统默认的 `/sys` 目录只有 `root` 用户才有写权限，本组的 ROS 程序如果以普通用户（`lckfb`）运行，去写 `export` 文件就会因为权限不足被拒绝。第二是操作顺序不对。在 Linux 中，控制 PWM 需要先往 `export` 写入通道号来启用它，然后再去设置周期（`period`）和占空比（`duty_cycle`），最后设置使能（`enable`）。如果顺序颠倒，系统会直接报错拒绝写入。

根据上述原因，对系统进行了以下调整与验证。1. 为了解决权限问题，本人修改了系统配置，或者在运行 ROS 节点前，用 `sudo chmod -R 777 /sys/class/pwm/` 临时放开 PWM 目录的权限。2. 本人重新理顺了 C++ 代码中的初始化逻辑：先检查 `/sys/class/pwm/pwmchip0/pwm0` 目录是否存在，如果不存在，先往 `export` 写 0 创建它；然后依次设置周期（对于 50Hz 的舵机，设置周期为 20000000 纳秒）；接着写入初始占空比；最后往 `enable` 写入 1 开启输出。3. 本人把这部分底层的 PWM 控

制代码封装成了一个独立的测试脚本。每次调舵机前，先用这个脚本让舵机转动，确认硬件和驱动没问题，再跟主程序联调。

程序触发投弹，但弹体没有释放

在该环节的首次联调中，出现的主要现象如下。在地面调试时，本人听到舵机“明显”一声转了，日志里也打印了“投弹成功”，但是挂在架子上的模型炸弹却依然仍停留在挂钩上，没有掉下来。

结合命令输出、节点状态和接口工作过程进行检查后，原因可归纳如下。舵机虽然动了，但不代表机械结构释放成功了。本人和硬件组的同学一起排查，发现原因有几个：一是舵机扫过的角度不够。本组在代码里设置的 PWM 占空比数值（即控制舵机转角）不够大，导致挂钩的锁销只是退出来了一半，还卡在挂钩孔里。二是供电问题。泰山派板子上的 5V 接口供电电流有限。当舵机挂载了有重量的炸弹模型时，启动随后电流过大，导致板子电压被拉低，舵机因供电不足中途卡死，甚至引起泰山派短暂死机。三是机械摩擦力太大，锁销和挂钩之间卡得太死。

根据上述原因，对系统进行了以下调整与验证。1. 首先本人和组员一起把舵机的供电线从泰山派上拔下来，改接到一个独立的 BEC 降压模块上，使用动力电池独立供电，确保舵机有足够的扭矩。2. 本人在代码里调整了控制占空比的数值，把开钩状态下的占空比微调得更大一些（比如从 1.5ms 对应占空比调整到 2.0ms），确保舵机能旋转到足够让锁销完全脱离的角度。3. 本组在地面进行了多次“负重挂载释放测

试”，手动往炸弹模型里塞入重物，模拟真实的重力环境，连续测试了十次都能顺利脱钩，这才算完成处理了机械释放的问题。

分级验证的实施过程

投弹代码难以直接进行实飞验证

在该环节的首次联调中，出现的主要现象如下。本人修改了自动投弹的控制算法，但是根本没法直接拿到飞机上飞来测试。因为实飞测试成本极高，需要申请场地，而且一旦飞控或者通信出点小问题，飞机掉下来摔坏了，整个组的实训就得受到影响。

结合命令输出、节点状态和接口工作过程进行检查后，原因可归纳如下。自动投弹是一个闭环控制过程，它需要：YOLO 跑通、定位算法正常、飞控回传速度高度、舵机正常动作。任何一个环节出错，实飞都会失败。如果在没有充分验证的情况下直接上机实测，不仅效率低，而且极其危险。

根据上述原因，对系统进行了以下调整与验证。为了安全，本人把验证过程拆成了几个地面仿真阶段：

1. 第一阶段：数据回放测试。本人把之前飞机实际飞行时的 sensor 数据和图像录成了 rosbag（数据包），在本地回放，然后运行本人的投弹节点，观察它算出来的投弹延迟、目标经纬度是否正常，把算法的逻辑错误在电脑上先改完。

2. 第二阶段：地面静态测试。本组把泰山派和舵机装在无人机机架上，飞机用架子固定在地面上。本人拿着一幅印有坦克的纸在摄像头前晃动，模拟飞机飞过目标。观察投弹命令触发时，舵机能否把挂钩甩开。
3. 第三阶段：低空手持测试。本组手持飞机在操场上小跑，模拟低速飞行，验证在有一定速度输入时，算法输出的延迟和释放动

作是否合理。通过这几步地面验证，把绝大多数代码 bug 都消灭在了地面，最后真正实飞的时候，一次投放就成功了。

4.17 外场实飞与实时回传

外场验收在东操场进行。起飞前完成机体固定、舵面方向、飞控状态、GPS、遥控器、任务载荷、电池和网络链路检查。地面虚拟机保持 WireGuard、ROS 环境和图像查看窗口运行，泰山派依次启动 MAVROS、摄像头、识别和任务节点。确认终端无持续错误、图像能够显示且遥控链路正常后，固定翼进行起飞。

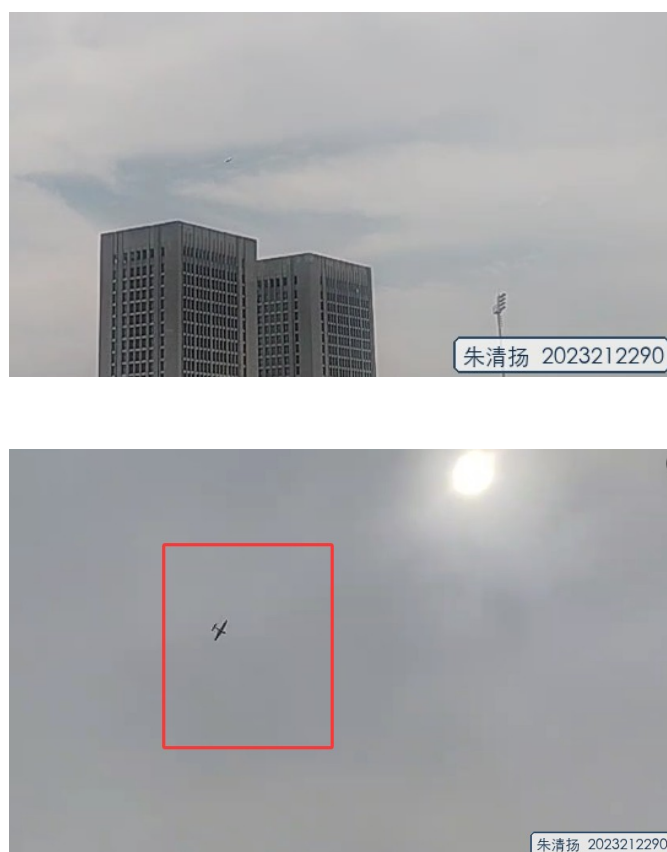


图 4.23 固定翼无人机起飞和空中飞行状态

连续截图显示飞机从建筑物附近升空，随后在操场上空保持飞行。图中红框标记飞机位置，能够反映飞机与地面背景的相对变化。

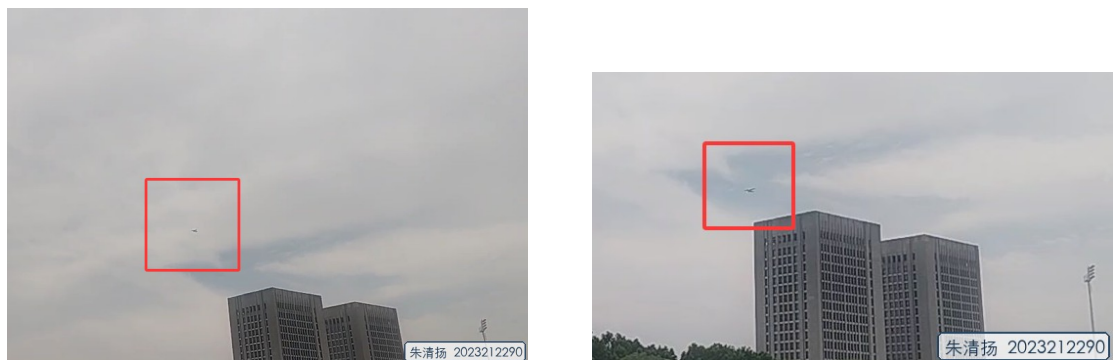


图 4.24 固定翼无人机外场飞行连续画面

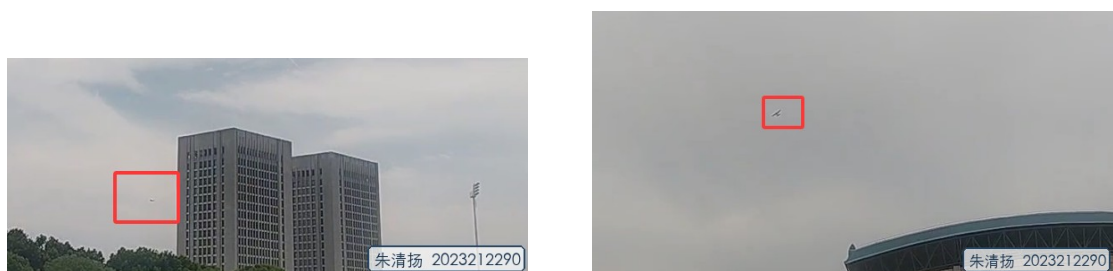


图 4.25 固定翼无人机航迹变化及远距离飞行画面

实飞过程中，地面虚拟机的 `rqt_image_view` 显示机载摄像头拍摄到的草地和机身局部，终端同时显示任务节点运行信息。该画面将飞机物理飞行、4G 网络、WireGuard 隧道、ROS 图像话题和地面显示连接在同一验证场景中。

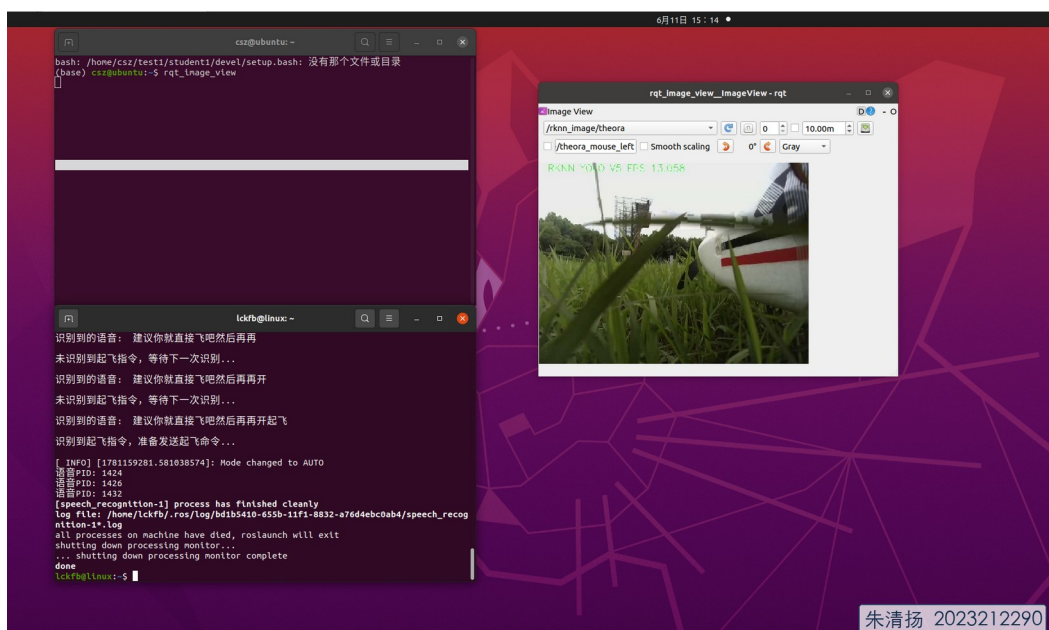


图 4.26 外场实飞过程中地面虚拟机接收的机载画面

4.18 综合测试结果与分析

综合测试从模型、网络、图传、任务程序和飞行平台五个方面评价系统。模型训练方面，exp5 的各项指标高于两组 100 轮训练，验证了更充分训练和最佳权重选择的作用；但 $mAP@0.5:0.95$ 仍低于 $mAP@0.5$ ，说明精细定位和复杂场景泛化需要继续改进。网络方面，WireGuard 解决了两个 NAT 客户端无法直接访问的问题，服务器转发和客户端保活保证了三端通信；校园网和 4G 的时延与丢包仍比局域网波动更大。

图传方面，压缩图像显著降低了带宽占用，使地面端能够持续显示机载画面。压缩同时会损失部分细节，因此识别推理应尽量在泰山派本地完成，地面图像主要用于监控和调试，不承担高速飞行控制闭环。任务程序方面，多帧聚合、任务区域、速度门限、高度有效性和单次触发锁定共同降低了误动作风险。执行机构方面，程序触发、PWM 输出、舵机动作和机械释放是四个不同的验收层级，必须分别确认。

表 4.6 系统主要方案对比

对比项目	初始方式	最终方式	改进效果
目标坐标	使用单帧检测结果	范围过滤与多帧平均	降低检测框波动造成的坐标跳变
图像传输	原始图像话题	压缩图像话题	降低 4G 上行带宽占用
ROS 地址	使用物理网卡地址	使用 WireGuard 地址	控制面与数据面均可跨公网访问
VPN 稳定性	仅建立基础隧道	转发、路由、保活和持久化	空闲和重启后的链路更稳定
投弹判断	直接按理论时间触发	状态门限、补偿和触发锁定	避免除零、无效数据和重复动作
舵机测试	与主任务同时调试	PWM 独立测试后再集成	区分程序、供电和机械问题

表 4.7 系统综合测试结论

测试对象	验证结果	对应证据
固定翼飞行	能够完成起飞和空中飞行	图 4.23 至图 4.25
YOLO 模型	exp5 指标达到本项目验证要求	表 4.3、图 4.10 至图 4.12
三端网络	云服务器、虚拟机和泰山派构成虚拟网段	图 4.14、相关配置与终端检查
4G 图传	地面虚拟机能够接收机载实时画面	图 4.16、图 4.26
语音任务	能够识别指令并推送航点、切换模式	图 4.17、图 4.18
投弹程序	具备坐标聚合、时机计算和 PWM 控制链路	图 4.19 至图 4.22 及关键代码

系统已经完成课程要求的综合功能闭环。后续提高任务精度时，需要进一步扩充目标数据集，使用规范相机标定获得内外参数，记录采集、推理、传输和舵机动作时间戳，并把风估计、姿态角和弹体阻力纳入投放模型。

5. 课程设计总结

本次课程设计完成了固定翼无人机飞行平台、机载嵌入式计算、目标识别、跨公网通信、实时图传、语音任务、目标定位和自动投弹控制的综合实现。项目不是将若干独立实验简单拼接，而是围绕“任务输入、航线执行、目标感知、位置解算、释放控制和地面监控”建立完整数据链。飞控、泰山派、摄像头、YOLO 模型、MAVROS、WireGuard、4G 网络和投弹舵机均具有独立的工作条件，只有接口、地址、数据单位、启动顺序和物理结构全部匹配时，系统才能稳定运行。

本人承担的主要工作包括 YOLO 目标识别模型训练与部署、自动投弹程序编写和逻辑调试、阿里云服务器与 WireGuard 虚拟专网配置、泰山派与虚拟机的 ROS 通信以及 4G 实时图传链路联调。在模型训练过程中，完成了数据集检查、训练参数配置、曲线分析和最佳权重选择；在网络调试过程中，完成了端口开放、地址规划、路由、内核转发、防火墙规则、NAT 保活和服务持久化；在 ROS 图传过程中，完成了

Master 地址、节点注册地址、图像消息类型和传输带宽的检查；在投弹程序中，完成了目标坐标聚合、速度与高度门限、自由落体时间、执行延迟补偿和 PWM 舵机控制。

本项目的主要特点是采用云服务器与 WireGuard 将移动网络后的泰山派和校园网后的虚拟机连接到同一虚拟网段，使 ROS 节点可以使用稳定地址完成跨公网通信；目标识别结果不仅用于图像显示，还与飞控位置、高度、航向和速度结合，用于目标地理坐标与释放时机计算；系统采用多帧聚合、地理范围、数据有效性和分级测试等安全约束，避免单帧误检或异常状态直接触发执行机构。

课程设计过程中也发现了若干不足。首先，目标数据集在场景、光照、视角、遮挡和目标尺度方面仍不充分，模型在严格交并比阈值下的定位能力还有提升空间。其次，目标坐标计算采用局部平面和相机垂直安装近似，尚未利用完整相机标定和飞机实时滚转、俯仰姿态。再次，投弹模型采用水平匀速和理想自由落体近似，对风、阻力和姿态变化的描述较为有限。最后，4G 链路受无线环境影响，地面图像适合监控和任务确认，但不适合作为高速姿态闭环输入。

后续改进可以从数据、定位、时延和弹道四个方面进行。数据方面增加难例、负样本和多尺度目标，比较不同轻量网络在 RK3566 上的精度与速度。定位方面使用棋盘格完成相机内参标定，建立相机、云台、机体和北东地坐标系之间的完整外参关系。时延方面在采集、推理、传输、判断和舵机动作处加入时间戳，形成端到端延迟模型。弹道方面融合飞控风估计、姿态和多次外场数据，对释放点进行在线修正。

通过本次实训，本人掌握了复杂系统的分层调试方法。网络故障按照端口、握手、路由、转发和业务顺序检查；ROS 故障区分 Master 控制面与话题数据面；模型故障区分数据、训练、转换、预处理和后处理；投弹故障区分数值计算、PWM 输出、舵机供电和机械释放。该方法使问题定位从反复重启和试错转变为基于状态、日志和接口的逐层验证，对后续嵌入式系统、机器人和网络工程实践具有直接作用。

6. 参考文献

Beard R. W., McLain T. W. Small Unmanned Aircraft: Theory and Practice[M]. Princeton: Princeton University Press, 2012.

Quigley M., Conley K., Gerkey B., et al. ROS: an open-source Robot Operating System[C]. ICRA Workshop on Open Source Software, 2009.

Redmon J., Divvala S., Girshick R., Farhadi A. You Only Look Once: Unified, Real-Time Object Detection[C]. IEEE Conference on Computer Vision and Pattern Recognition, 2016: 779-788.

Jocher G., Chaurasia A., Stoken A., et al. ultralytics/yolov5: v7.0 - YOLOv5 SOTA Realtime Instance Segmentation[CP/OL]. Zenodo, 2022.

Donenfeld J. A. WireGuard: Next Generation Kernel Network Tunnel[R]. 2017.

Meier L., Tanskanen P., Heng L., et al. PIXHAWK: A micro aerial vehicle design for autonomous flight using onboard computer vision[J]. Autonomous Robots, 2012, 33: 21-39.

Hartley R., Zisserman A. Multiple View Geometry in Computer Vision[M]. Cambridge: Cambridge University Press, 2004.

Szeliski R. Computer Vision: Algorithms and Applications[M]. Cham: Springer, 2022.

Ultralytics. YOLOv5 Repository and Documentation[EB/OL]. <https://github.com/ultralytics/yolov5>.

ArduPilot Development Team. ArduPilot Plane and Mission Planner Documentation[EB/OL]. <https://ardupilot.org/>.

ROS Community. ROS Wiki and MAVROS Documentation[EB/OL]. <https://wiki.ros.org/mavros>.

WireGuard Project. WireGuard Documentation[EB/OL]. <https://www.wireguard.com/>.